

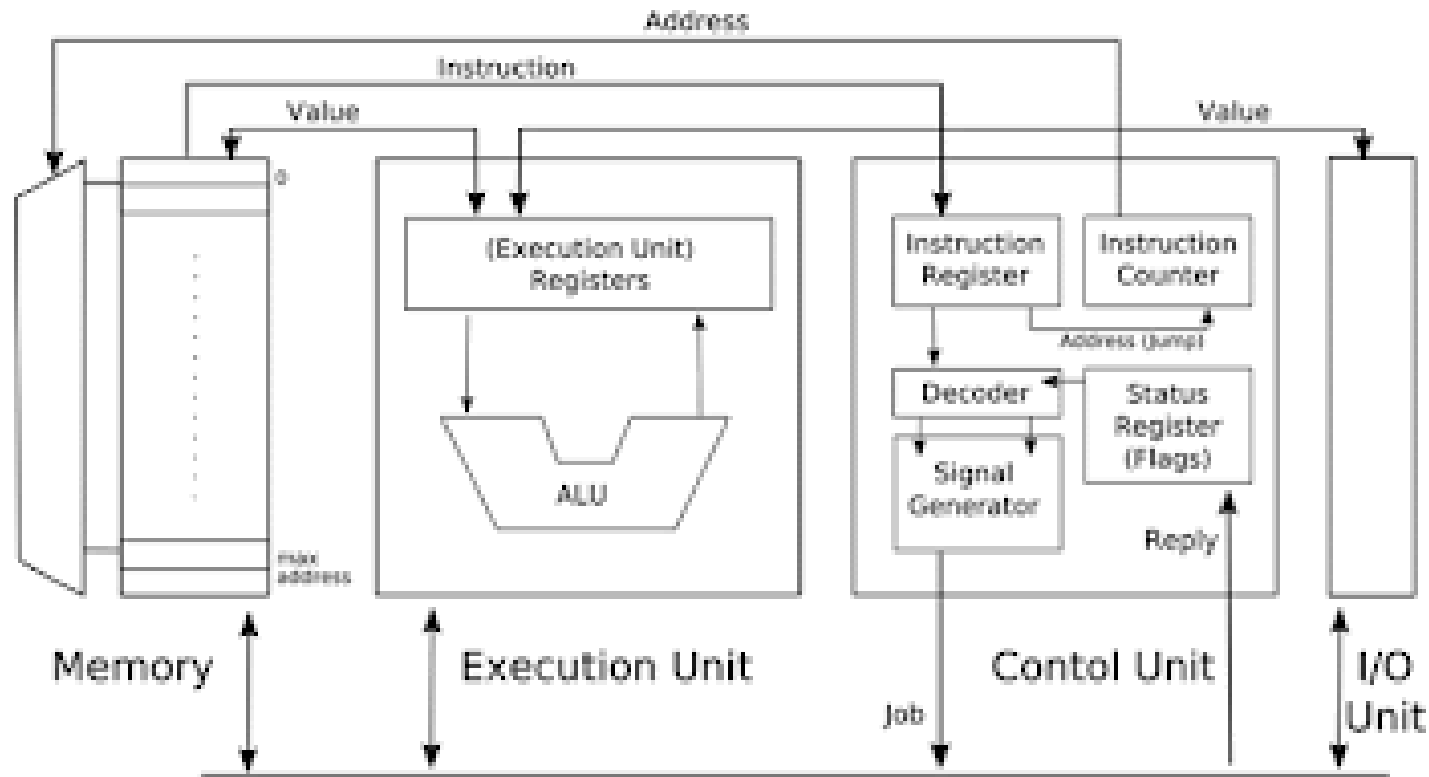
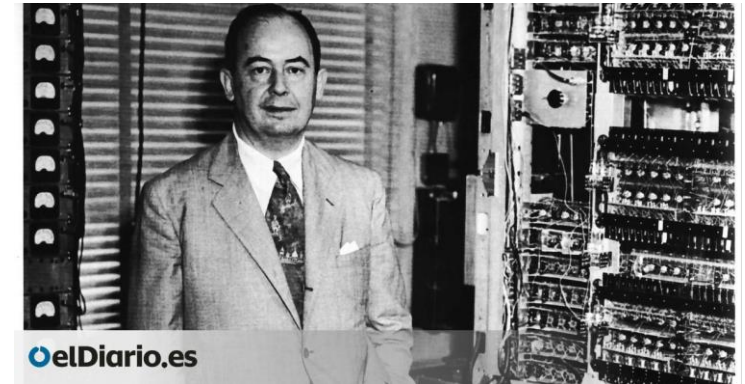
Single Processor Computing

And a first set of basic concepts every computer scientists should know

Introduction

- While to some scientists the abstract fact of the existence of a solution is enough, in HPC we actually want that solution, and preferably yesterday
 - efficiency of both algorithms and hardware
 - arithmetic precision
 - the difference between two algorithmic approaches can enable hardware-specific optimizations
- Therefore, in order to write efficient scientific codes, it is important to understand your computer architecture.
- The difference in speed between two codes that compute the same result can range from a few percent to orders of magnitude, depending only on factors relating to how well the algorithms are coded for the processor architecture.

The Von Neumann Architecture



Bus

<http://abelgo.cn/cs101/papers/Neumann.pdf>

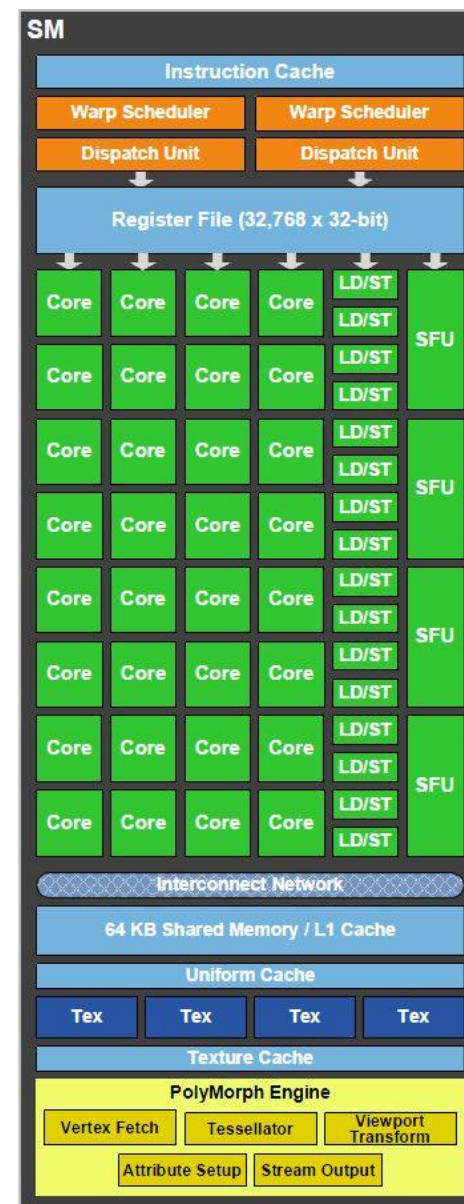
Example

- Many NVIDIA GPUs are equipped with Special Function Units (SFUs) that execute transcendental/intrinsics instructions such as sin, cosine, reciprocal, and square root
- PROS: each SFU executes one transcendental instruction per thread, per clock, wrt to the traditional instructions that require several FMAD instructions
- CONS: limited precision

Algorithm	GFLOPs
CUDA	167.71
OpenACC	147.57
CUDA - fastmath	434.46
OpenACC - fastmath	211.80
CUDA reference	597.11

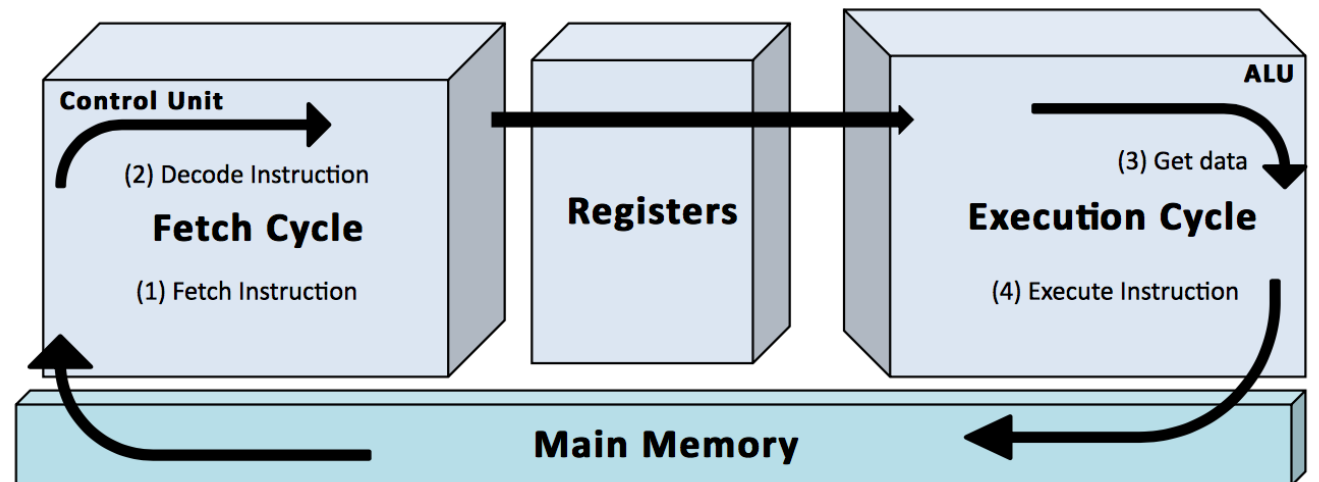
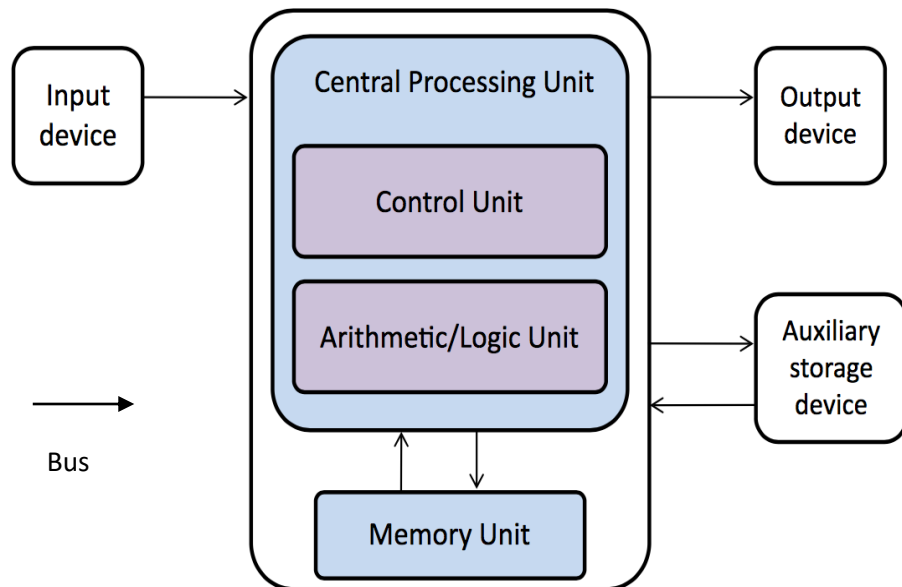
Function	Error bounds
<code>__sinf(x)</code>	For x in $[-\pi, \pi]$, the maximum absolute error is $2^{-21.41}$, and larger otherwise.
<code>__cosf(x)</code>	For x in $[-\pi, \pi]$, the maximum absolute error is $2^{-21.19}$, and larger otherwise.
<code>__sincosf(x, sptr, cptr)</code>	Same as <code>__sinf(x)</code> and <code>__cosf(x)</code> .
<code>__tanf(x)</code>	Derived from its implementation as <code>__sinf(x) * (1 / __cosf(x))</code> .
<code>__powf(x, y)</code>	Derived from its implementation as <code>exp2f(y * __log2f(x))</code> .

The Fermi Architecture



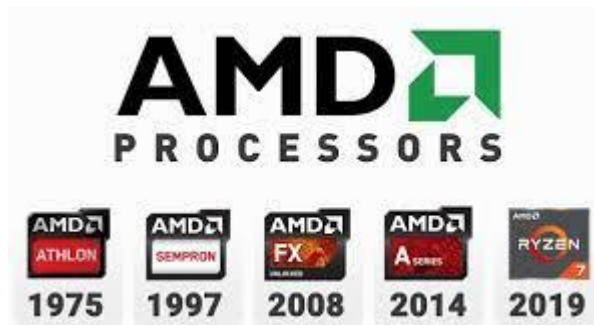
von Neumann Architecture

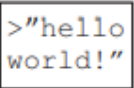
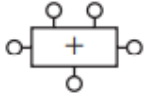
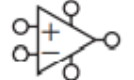
- Present computers have many aspect in common.
- Their architecture is based on an undivided memory that stores both program and data ('stored program'), and a processing unit that executes the instructions, operating on the data in 'fetch, execute, store cycle'
- We can say that a computer is the HW implementation of that cycle, and it represents an universal executor of algorithms



Architectures

- But, actually, a computer architecture (e.g. x86) is defined by its
 - instruction set
 - architectural state, i.e. the information necessary to define what a computer is doing (PC + registers).
- A microarchitecture involves combining logic elements to execute the instructions defined by the architecture



Application Software		Programs
Operating Systems		Device Drivers
Architecture		Instructions Registers
Micro-architecture		Datapaths Controllers
Logic		Adders Memories
Digital Circuits		AND Gates NOT Gates
Analog Circuits		Amplifiers Filters
Devices		Transistors Diodes
Physics		Electrons

Processing instructions

```
void store(double *a, double *b, double *c) {  
    *c = *a + *b;  
}
```

gcc -O2 -S -o - store.c or, better

gcc -O2 -g -Wa,-adhln -fverbose-asm store.c > store.s

store:

```
movsd (%rdi), %xmm0 # Load *a to %xmm0  
addsd (%rsi), %xmm0 # Load *b and add to %xmm0  
movsd %xmm0, (%rdx) # Store to *c  
ret
```

Each instruction is processed as follows:

- Fetch instruction from memory.
- Decode the instruction and read registers
- Execute the operation OR calculate an address
- Access an operand in data memory (if necessary)
- write the result into a register (if necessary)

Slide on x86 Assembler

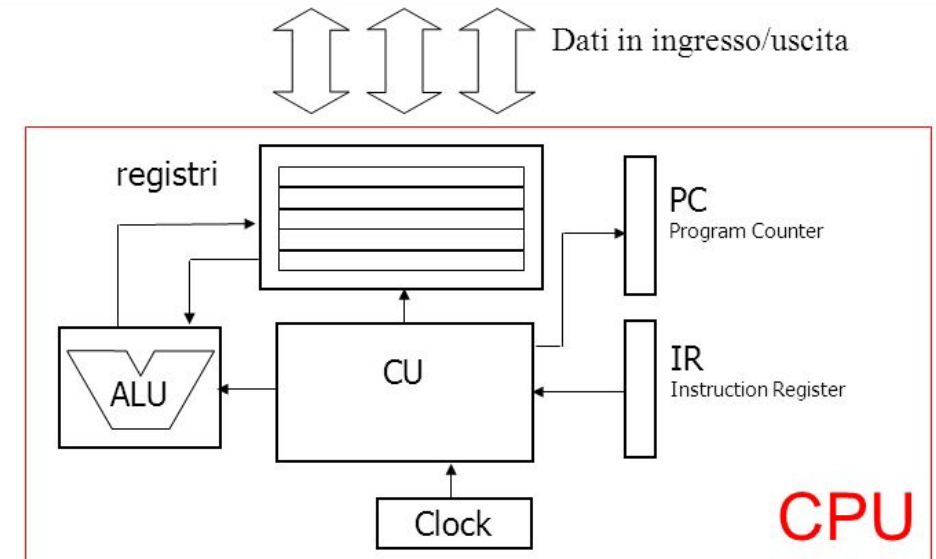
<https://brown-cs0330.github.io/website/index.html>

Compiler explorer

<https://godbolt.org>

A bit of assembler <https://stackoverflow.com/questions/5325326/what-is-the-meaning-of-each-line-of-the-assembly-output-of-a-c-hello-world/38290148>

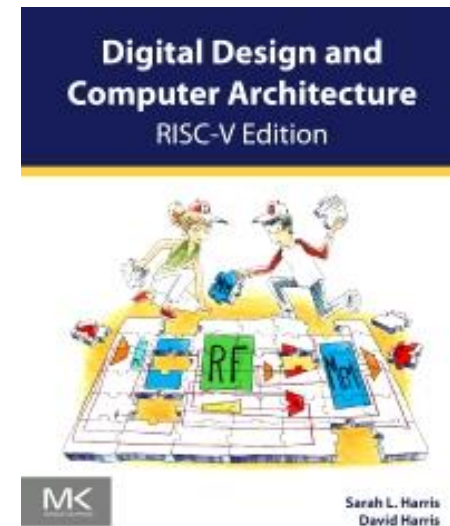
BUS



Risc-V Architecture

Table 6.1 RISC-V register set

Name	Register Number	Use
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporary registers
s0/fp	x8	Saved register/Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments/Return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporary registers



Code Example 6.21 USING A FOR LOOP TO ACCESS AN ARRAY

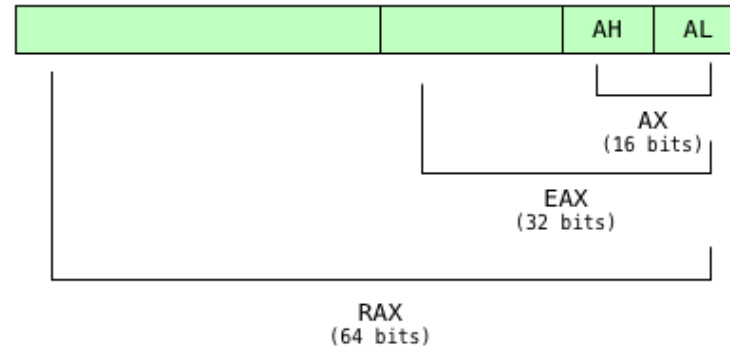
High-Level Code

```
int i;  
int scores[200];  
  
for (i = 0; i < 200; i = i + 1)  
  
    scores[i] = scores[i] + 10;
```

RISC-V Assembly Code

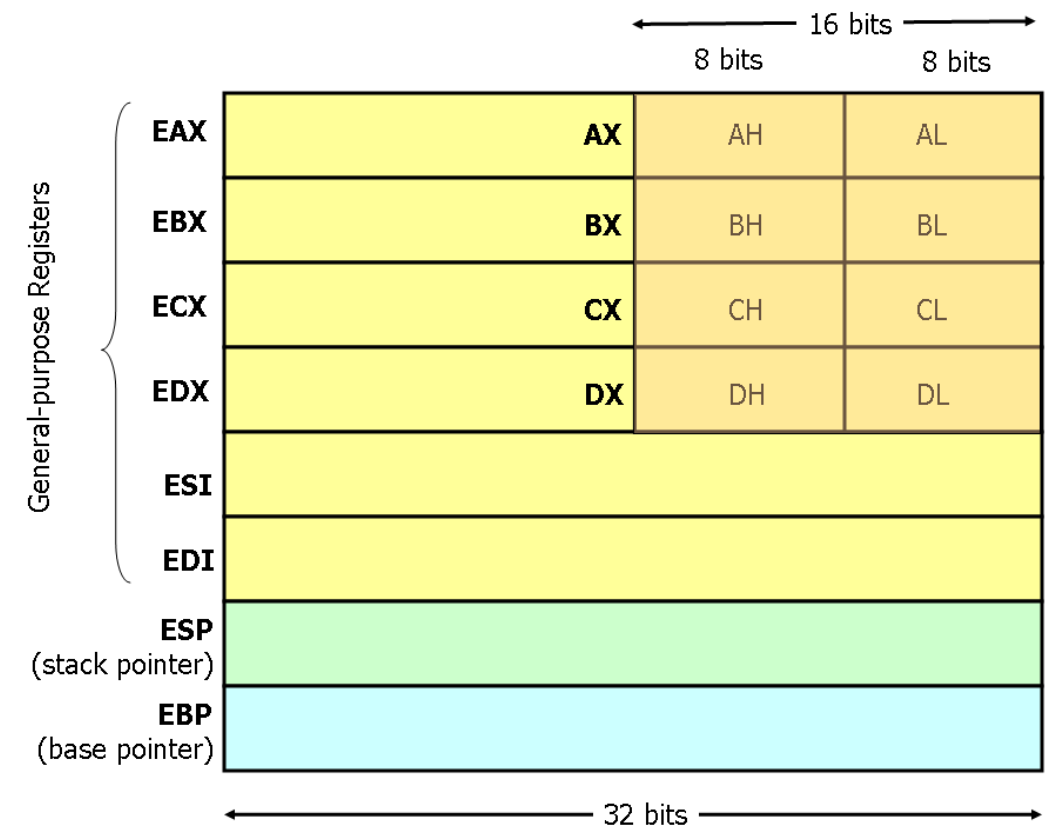
```
# s0 = scores base address, s1 = i  
  
addi s1, zero, 0    # i = 0  
addi t2, zero, 200  # t2 = 200  
  
for:  
    bge s1, t2, done # if i >= 200 then done  
    slli t0, s1, 2    # t0 = i * 4  
    add t0, t0, s0    # address of scores[i]  
    lw t1, 0(t0)      # t1 = scores[i]  
    addi t1, t1, 10    # t1 = scores[i] + 10  
    sw t1, 0(t0)      # scores[i] = t1  
    addi s1, s1, 1    # i = i + 1  
    j for             # repeat  
done:
```

X86 Registers



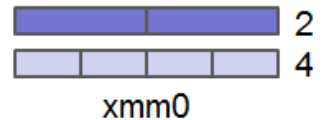
- Small amount of memory internal to a processor
- The registers are what the processor actually operates on
- An operation like $a = b + c$ involves three registers
- Registers have specific names, sizes, a high bandwidth and low latency because they are part of the processor.

<https://www.cs.virginia.edu/~evans/cs216/guides/x86.html>

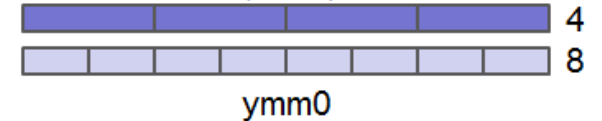


64-bit double
32-bit float

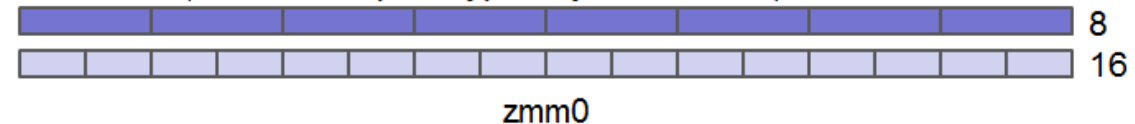
SSE, 128-bit (1999)



AVX, 256-bit (2011)



AVX-512 (KNL, 2016; prototyped by KNC, 2013)



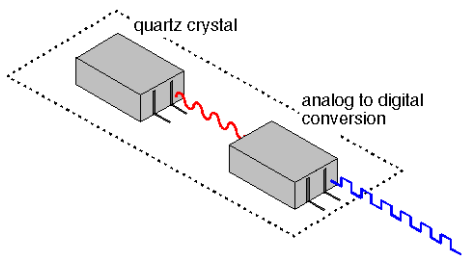
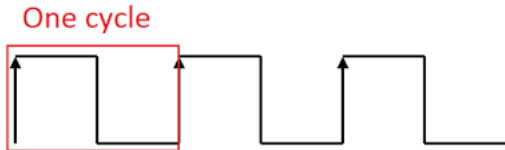
The Clock

The instructions of all modern processors need to be *synchronised* with a timer or *clock*.

The *clock cycle* τ is defined as the time between two adjacent pulses of oscillator that sets the time of the processor.

The number of these pulses per second is known as *clock speed* or *clock frequency*, generally measured in GHz (gigahertz, or billions of pulses per second).

The clock cycle controls the synchronization of operations in a computer: All the operations inside the processor last a multiple of τ .



The quartz crystal generates continuous waves, which are converted into digital pulses.

Processor	τ (ns)	freq (MHz)
CDC 6600	100	10
Cyber 76	27.5	36
IBM ES 9000	9	111
Cray Y-MP C90	4.1	244
Intel i860	20	50
PC Pentium	< 0.5	> 2 GHz
Power PC	1.17	850
IBM Power 5	0.52	1.9 GHz
IBM Power 6	0.21	4.7 GHz

Increasing the clock frequency:

The *speed of light* sets an upper limit to the speed with which electronic components can operate .

Propagation velocity of a signal in a vacuum:
300,000 Km/s = 30 cm/ns

Heat dissipation problems inside the processor. Power consumption varies as the square or cube of the clock frequency.

edge-triggered clocking

- Any values stored in a logic element are updated only on a clock edge, which is a quick transition from low to high or vice versa
- The inputs are values that were written in a previous clock cycle, while the outputs are values that can be used in a following clock cycle.

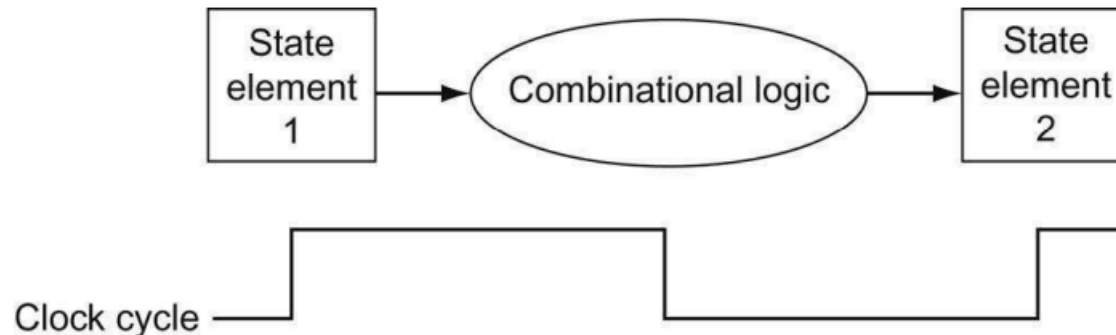


FIGURE 4.3 Combinational logic, state elements, and the clock are closely related.

CPU performance

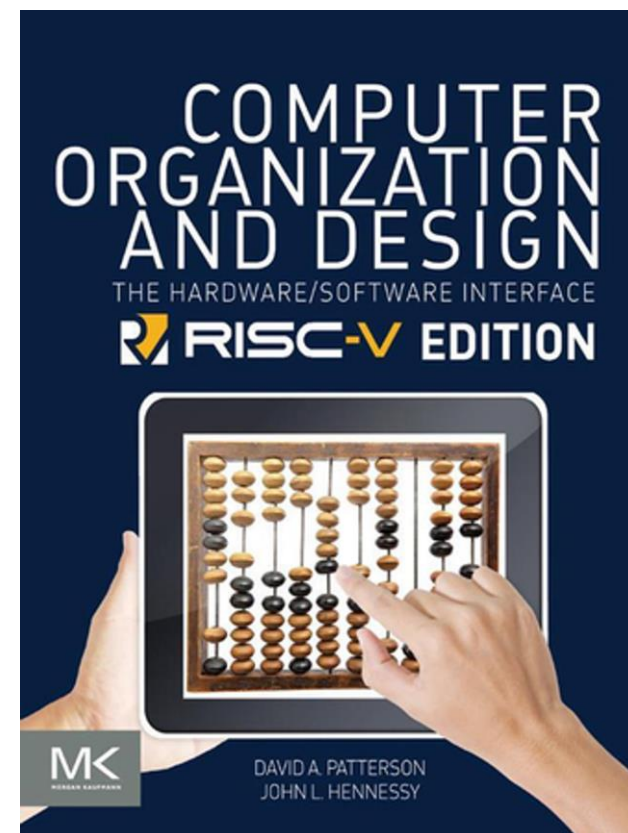
- Execution time: sum of the actual cycles required for each specific instruction
https://www.agner.org/optimize/instruction_tables.pdf
- Execution time: (number of instructions x average CPI) / clock frequency
- A program is converted in
 - 1) 4 billion MIPS instructions. The MIPS processor requires, on average, 1.5 Cycles Per Instruction and the clock speed is 1 GHz
 - 2) 2 billion x86 instructions. The x86 processor requires, on average, 6 CPI and the clock speed is 1.5 GHzWhich is better?

<https://www.anandtech.com/show/3593>

Understanding Program Performance

The performance of a program depends on the algorithm, the language, the compiler, the architecture, and the actual hardware. The following table summarizes how these components affect the factors in the CPU performance equation.

Hardware or software component	Affects what?	How?
Algorithm	Instruction count, CPI	The algorithm determines the number of source program instructions executed and hence the number of processor instructions executed. The algorithm may also affect the CPI, by favoring slower or faster instructions. For example, if the algorithm uses more divides, it will tend to have a higher CPI.
Programming language	Instruction count, CPI	The programming language certainly affects the instruction count, since statements in the language are translated to processor instructions, which determine instruction count. The language may also affect the CPI because of its features; for example, a language with heavy support for data abstraction (e.g., Java) will require indirect calls, which will use higher CPI instructions.
Compiler	Instruction count, CPI	The efficiency of the compiler affects both the instruction count and average cycles per instruction, since the compiler determines the translation of the source language instructions into computer instructions. The compiler's role can be very complex and affect the CPI in varied ways.
Instruction set architecture	Instruction count, clock rate, CPI	The instruction set architecture affects all three aspects of CPU performance, since it affects the instructions needed for a function, the cost in cycles of each instruction, and the overall clock rate of the processor.



Moore's Law

- In 1965, Gordon Moore proposed that the number of transistors on a silicon chip would double every year.
- Moore's Law, as it is now known, proved prophetic about the exponential growth of computing power that made much of the modern world possible.
- A transistor produces, amplifies, and directs an electrical signal using three leads, a source, a gate, and a drain.
- When voltage is applied to the gate lead, an incoming current at the source lead will be allowed to pass through to the drain lead. Take the voltage away from the gate lead and the current cannot pass through.
- What this does is produce a way to compute logical values, 1 and 0 in computer terms, based on the whether there is voltage applied to the gate and the source leads.
- Connect the drain lead of a transistor to the source lead or the gate lead of another transistor and suddenly you can start producing incredibly complex logic systems. **And the more transistors you can fit on a chip, the more computationally powerful this network becomes.**

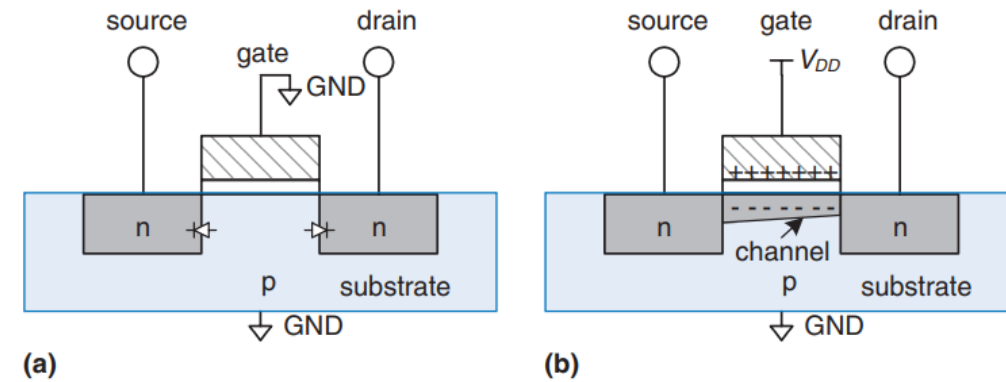
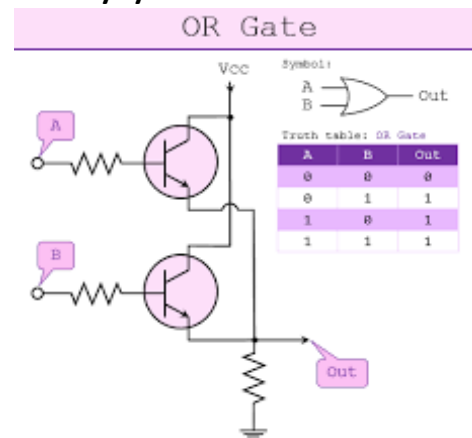


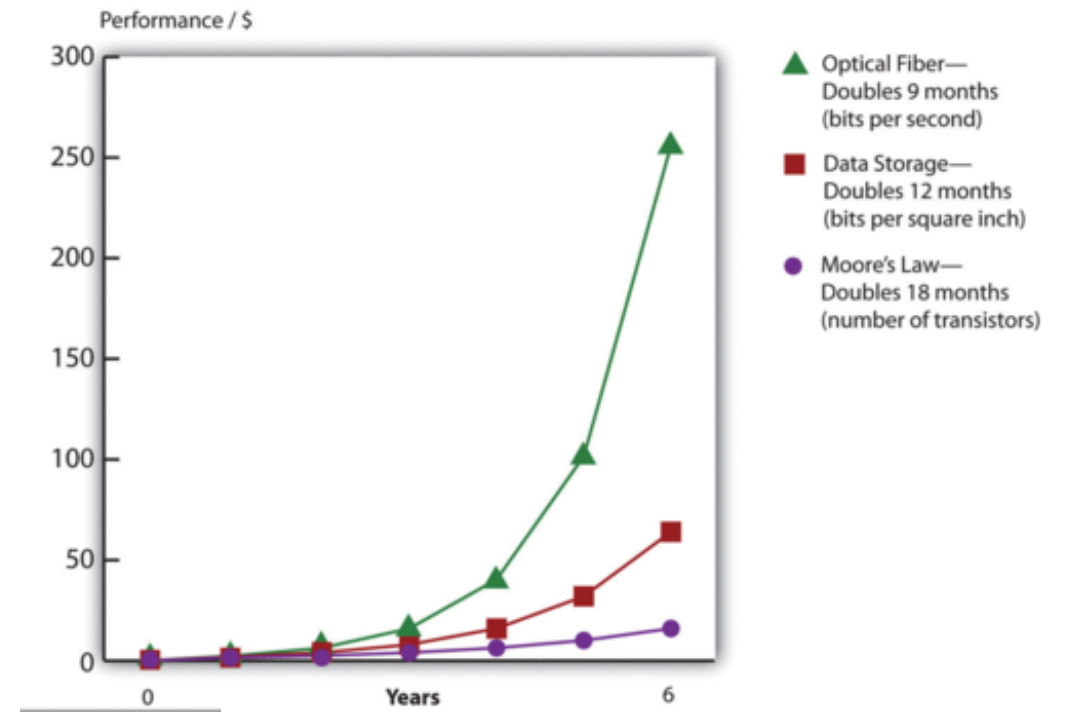
Figure 1.30 nMOS transistor operation

<http://www.moorelaw.org> How a CPU is made <https://www.youtube.com/watch?v=qm67wbB5GmI>
<https://interestingengineering.com/no-more-transistors-the-end-of-moores-law>
<https://www.101computing.net/from-transistors-to-micro-processors/>



Other formulations and atoms

- Butters' law says that the amount of data coming out of an optical fiber is doubling every nine months.
- Kryder's Law says that «Inside of a decade and a half, hard disks had increased their capacity 1,000-fold.

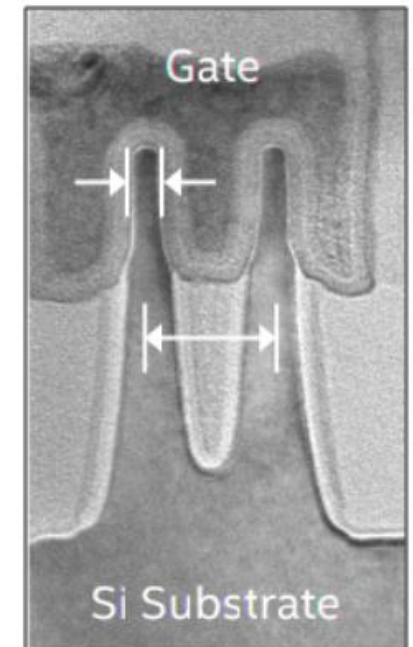


The **atoms** used in silicon chip fabrication are around 0.2nm. CPU Vendors measure their technology in nm, but that's not the size of individual transistors; that's actually a measure of the distance *between* discrete components on a chip.

<https://www.extremetech.com/computing/97469-is-14nm-the-end-of-the-road-for-silicon-lithography>

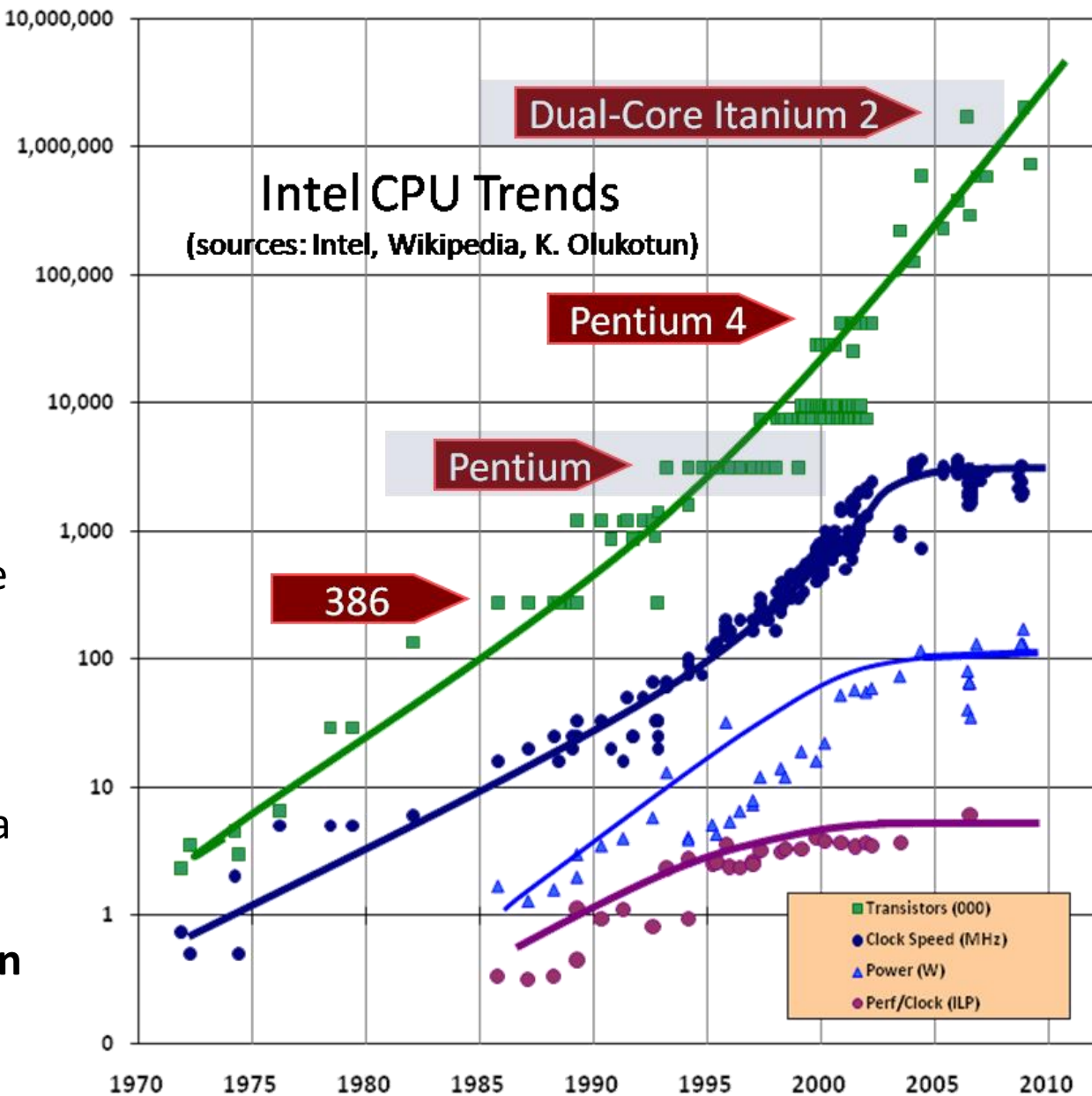
8 nm Fin Width

42 nm Fin Pitch



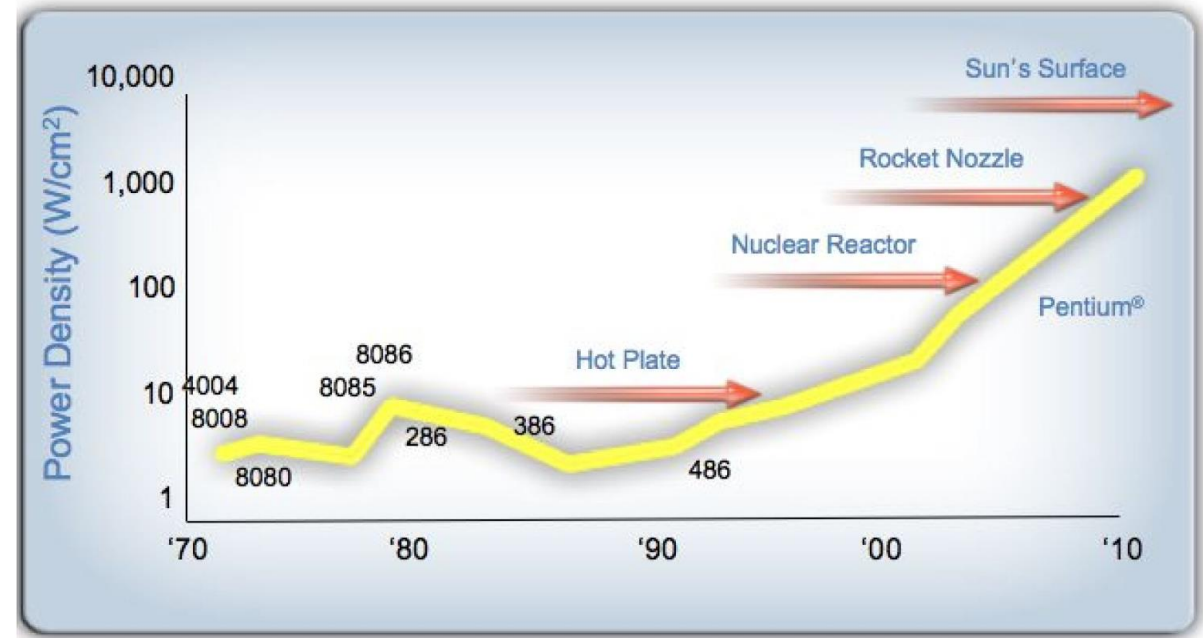
Moore's Law

- The speeds have barely doubled within a 10 year span (i.e. 2000-2010).
- This is because we are looking at the speeds and not the **number of transistors**.
- The speeds over a large number of years for example went between 1.3 and 2.8 GHz, which is barely double but what needs to be kept in mind is that the 2.8 is a QUAD CORE while the 1.3 is a single CORE.
- This means that the actual power of the 2.8 would be found if you **multiply by four** – which would give you a whopping 11.2 which is a far cry from 1.3.
- **The limitation which exists is that once transistors can be created as small as atomic particles, then there will be no more room for growth in the CPU market where speeds are concerned**



Moore's Law is Dead. Long Live Moore's Law!

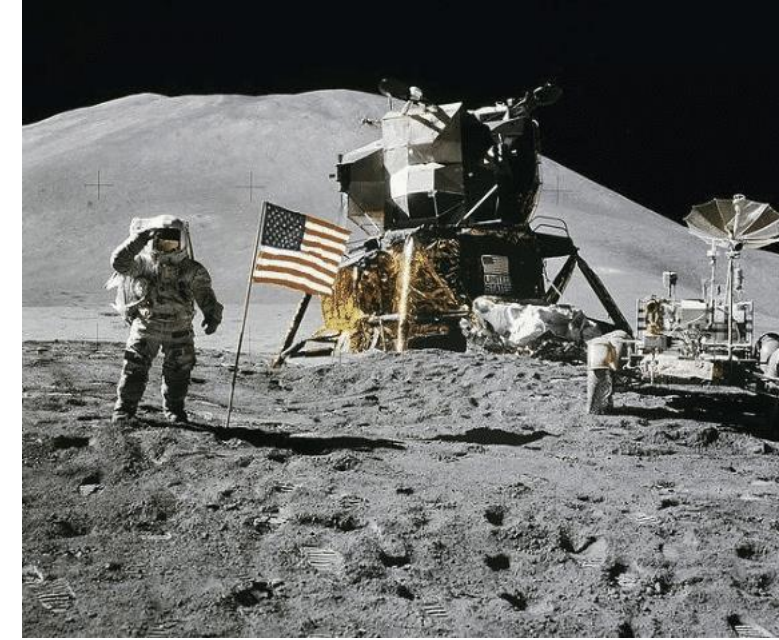
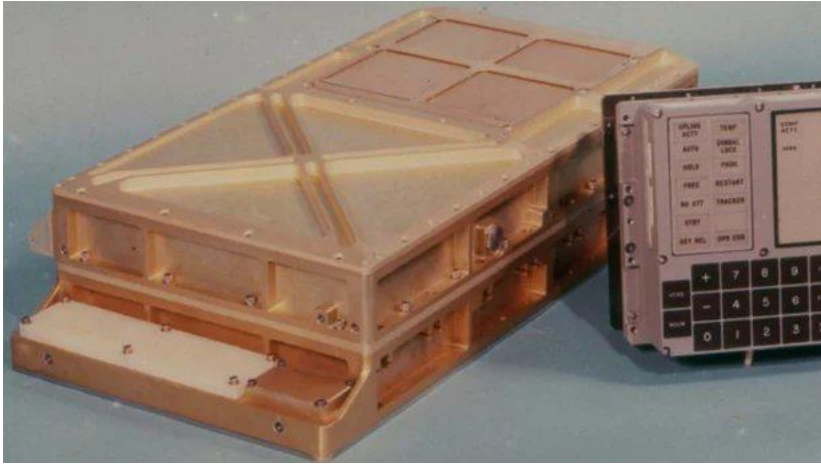
- The frequency of a single core can not be scaled up since this would raise the heat production of the chip too far.
 - They have gotten so small and the channel that carries the electrical current through the transistor cannot always contain it. This generates heat.
 - When the number of transistors doubles, so does the amount of heat they can generate.
 - The cost of cooling is becoming too big.
- The graph shows the heat that a chip would give off, if single-processor trends had continued.
- The computer design run into a power wall , where the sophistication of a single core can not be increased any further (we will see it, e.g. ILP) and **the only way to increase performance is to increase the amount of explicitly visible parallelism**
 - Each core is less powerful than the previous generation's single core design but their number increases



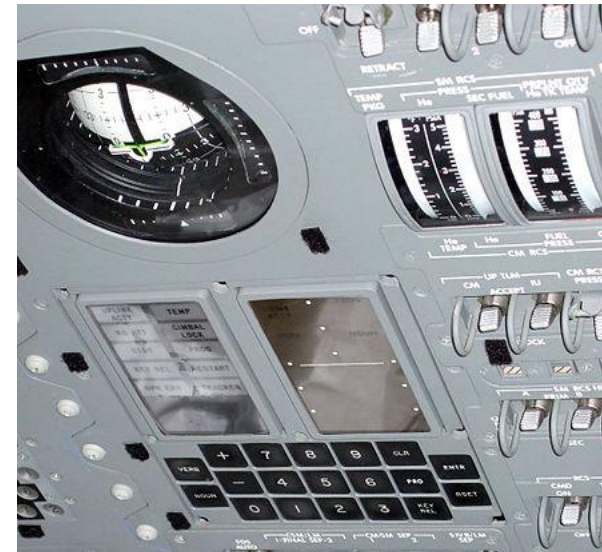
The Free Lunch Is Over

A Fundamental Turn Toward Concurrency in Software

- The biggest sea change in software development since the OO revolution is knocking at the door, and its name is Concurrency. (March 2005)
<http://www.gotw.ca/publications/concurrency-ddj.htm>
- Make a CPU ten times as fast, and software will usually find ten times as much to do (*or, in some cases, will feel at liberty to do it ten times less efficiently – see next slide*).
- Most classes of applications have enjoyed free and regular performance gains for several decades, even without releasing new versions or doing anything special, because the CPU manufacturers (primarily) and memory and disk manufacturers (secondarily) have reliably enabled ever-newer and ever-faster mainstream systems.
- Right enough, in the past. But dead wrong for the foreseeable future. **So?**



Apollo Guidance computer, 2 MHz, 72 KB RAM, 32 KG and about 1 MB di ROM.



The Free Lunch Is Over

Over the past 30 years, CPU designers have achieved performance gains in three main areas:

1. **Clock speed**

Increasing clock speed is about getting more cycles. Running the CPU faster more or less directly means doing the same work faster.

2. **Execution optimization**

Optimizing execution flow is about doing more work per cycle. Today's CPUs sport some more powerful instructions, and they perform optimizations that range from the pedestrian to the exotic, including pipelining, branch prediction, executing multiple instructions in the same clock cycle(s), and even reordering the instruction stream for out-of-order execution. These techniques are all designed to make the instructions flow better and/or execute faster, and to squeeze the most work out of each clock cycle by reducing latency and maximizing the work accomplished per clock cycle.

3. **Cache**

the memory is too slow to load data into the process at the rate the processor can absorb it. Specifically, a single load can take 1000 cycles, while a processor can perform several operations per cycle, therefore huge efforts are paid to reduce the so-called memory wall

This set of slides focuses on 2-3

The Free Lunch Is Over

Present performance growth drivers are:

- **Hyperthreading**

Hyperthreading is about running two or more threads in parallel inside a single CPU by better exploiting it when some threads wait the completion of I/O operations . A limiting factor, however, is that although a hyper-threaded CPU has some extra hardware including extra registers, it still has just one cache, one integer math unit, one FPU, and in general just one each of most basic CPU features. Hyperthreading is sometimes cited as offering a 5% to 15% performance boost for reasonably well-written multi-threaded applications, or even as much as 40% under ideal conditions for carefully written multi-threaded applications. That's good, but it's hardly double, and it doesn't help single-threaded applications.

- **Multicore**

Multicore is about running two or more actual CPUs on one chip. The performance gains ideally are about the same as having a true dual-CPU system (only the system will be cheaper because the motherboard doesn't have to have two sockets and associated "glue" chipper) for reasonably well-written multi-threaded applications. Not single-threaded ones.

- **Cache**

On-die cache sizes can be expected to continue to grow. Of these three areas, only this one will broadly benefit most existing applications. The continuing growth in on-die cache sizes is an incredibly important and highly applicable benefit for many applications, simply because space is speed.

- **Multicore(s) and related parallel programming techniques represent the main topic of this course**

Myths and Realities: $2 \times 3\text{GHz} < 6\text{ GHz}$

- So a dual-core CPU that combines two 3GHz cores practically offers 6GHz of processing power. Right? **Wrong.**
- **FIRST: memory**
Even having two threads running on two physical processors doesn't mean getting two times the performance. Similarly, most multi-threaded applications won't run twice as fast on a dual-core box. They should run faster than on a single-core CPU; the performance gain just isn't linear, that's all.
- Why not? First, there is coordination overhead between the cores to ensure cache coherency (a consistent view of cache, and of main memory) and to perform other handshaking. Today, a two- or four-processor machine isn't really two or four times as fast as a single CPU even for multi-threaded applications. The problem remains essentially the same even when the CPUs in question sit on the same die.
- **Second: coding**
Unless the two cores are running different processes, or different threads of a single process that are well-written to run independently and almost never wait for each other, they won't be well utilized.
- (Despite this, I will speculate that today's single-threaded applications as actually used in the field could actually see a performance boost for most users by going to a dual-core chip, not because the extra core is actually doing anything useful, but because it is running the adware and spyware that infest many users' systems and are otherwise slowing down the single CPU that user has today. I leave it up to you to decide whether adding a CPU to run your spyware is the best solution to that problem.)
- If you're running a single-threaded application, then the application can only make use of one core. There should be some speedup as the operating system and the application can run on separate cores, but typically the OS isn't going to be maxing out the CPU anyway so one of the cores will be mostly idle. (Again, the spyware can share the OS's core most of the time.)

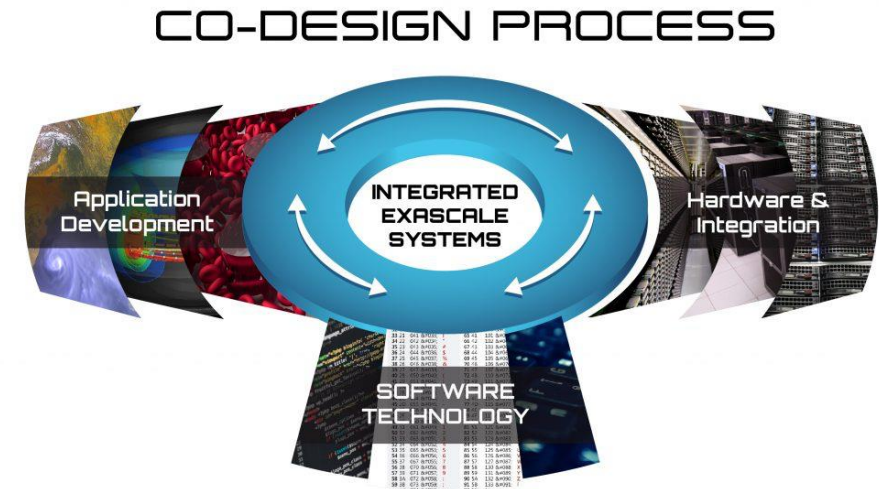
Concurrency is not new

- “Concurrency? That’s not news! People are already writing concurrent applications.” That’s true. Of a small fraction of developers.
- People have been doing object-oriented programming since at least the days of Simula in the late 1960s. But OO didn’t become a revolution, and dominant in the mainstream, until the 1990s. Why then? The reason the revolution happened was primarily that our industry was driven by requirements to write larger and larger systems that solved larger and larger problems and exploited the greater and greater CPU and storage resources that were becoming available. OOP’s strengths in abstraction and dependency management made it a necessity for achieving large-scale software development that is economical, reliable, and repeatable.
- A similar situation occurs today. Applications will increasingly need to be concurrent if they want to fully exploit CPU throughput gains that have now started becoming available and will continue to materialize over the next several years.
- Probably the key issue today is that **concurrency really is hard**: The programming model, meaning the model in the programmer’s head that he needs to reason reliably about his program, is much harder than it is for sequential control flow.

Conclusions

- Now is the time to take a hard look at the design of your application, determine what operations are CPU-sensitive now or are likely to become so soon, and identify how those places could benefit from concurrency.
- A few rare classes of applications are naturally parallelizable, but most aren't. Even when you know exactly where you're CPU-bound, you may well find it difficult to figure out how to parallelize those operations; all the most reason to start thinking about it now. Implicitly parallelizing compilers can help a little, but don't expect much; they can't do nearly as good a job of parallelizing your sequential program as you could do by turning it into an explicitly parallel and threaded version.
- Not all applications (or, more precisely, important operations of an application) are amenable to parallelization. The usual example here is that just because it takes one woman nine months to produce a baby doesn't imply that nine women could produce one baby in one month.
- But did you notice the problem with leaving the analogy at that? Here's the trick question to ask the next person who uses it on you: Can you conclude from this that the Human Baby Problem is inherently not amenable to parallelization? Usually people relating this analogy err in quickly concluding that it demonstrates an inherently nonparallel problem, but that's actually not necessarily correct at all.
- **It is indeed an inherently nonparallel problem if the goal is to produce one child. It is actually an ideally parallelizable problem if the goal is to produce many children!**
- Knowing the real goals can make all the difference. This basic goal-oriented principle is something to keep in mind when considering whether and how to parallelize your software.

Moore's law is dead – long live AI



- Nvidia founder and CEO, Jensen Huang
Accelerated computing looks at the entire stack, algorithms, software and processor,” he said. “We can study where bottlenecks are. New software systems make the application go faster, not just the chip”.
- “Let’s say you have an airplane that has to deliver a package. It takes 12 hours to deliver it. Instead of making the plane go faster, concentrate on how to deliver the package faster, look at 3D printing at the destination,” he explained. Applications acceleration is not just to make the chip go faster, it is to deliver the goal faster.”
- <https://www.electronicshweekly.com/news/products/micros/jensen-huang-moores-law-dead-long-live-ai-2018-10/>
- <https://siliconangle.com/2021/04/10/new-era-innovation-moores-law-not-dead-ai-ready-explode/>
- *But in this course we'll focus on traditional HPC techniques.*

von Neumann Architecture

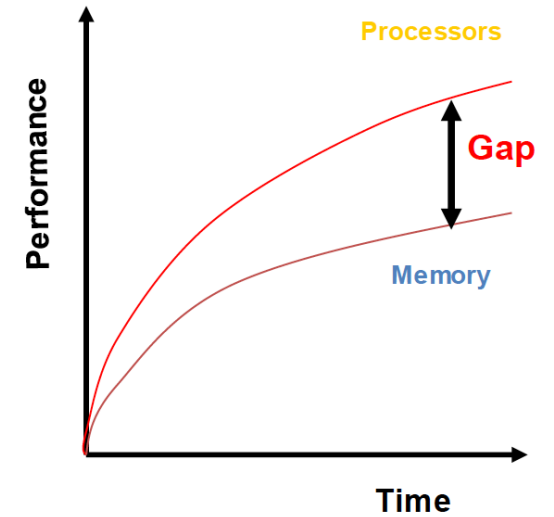
It is a too high level and simple abstraction for present computers.

- Memories

- The real/main limitation for a CPU is the performance difference between data to/from memory.
- while memory looks randomly addressable, in practice there is a concept of **locality**: once a data item has been loaded, nearby items are more efficient to load
- moreover, there are different **memory levels** to deal with the memory wall

- Instruction Level Parallelism (ILP)

- contemporary CPUs operate on several instructions **simultaneously** and, possibly, they are handled in a **different order** with respect to the sequential code
- while each instruction can take several clock cycles to complete, a processor can complete **one instruction per cycle** in favourable circumstances (pipelining)
- their inputs and outputs are also being moved in an **overlapping** manner



Pipelining

- A single instruction takes several clock cycles to complete
- Subdivide an instruction into stages (or segments):
 1. Instruction decode (including finding the location of operands)
 2. Copying the operands into registers ('data fetch').
 3. Aligning the exponents; the addition $.35 \times 10^{-1} + .6 \times 10^{-2}$ becomes $.35 \times 10^{-1} + .06 \times 10^{-1}$
 4. Actual operation
 5. Normalize the result if needed ($.41 \times 10^{-1}$ does not require it)
 6. Storing the result
- Pipeline: separate piece of hardware for each subdivision

Pipelining

- If every stage is designed to finish in 1 clock cycle, the whole instruction takes 5 cycles. However, if each stage has **its own hardware**, we can execute two operations in much less than 5 cycles:
 - Execute the IF for the first operation;
 - Do the ID for the first operation, and at the same time the IF for the second.
 - Execute the third stage for the first operation and the second stage of the second operation simultaneously.
 - ...
- You see that the first operation still takes 5 clock cycles, but the second one is finished a mere 1 cycle later, so both in 6 cycles.
- On a **traditional EU**, producing **n** results takes **$t(n) = nst$** where **s** is the number of stages, and **t** the clock cycle time.
- On a **pipelined EU** the time is **$t(n) = [s+(n-1)]$** where **s** becomes a setup cost: the first operation still must go through the same stages as before, but after that one result will be produced each cycle.

Instruction	Clock cycle								
	1	2	3	4	5	6	7	8	9
Instruction x	IF	ID	EXE	MEM	WB				
Instruction x+1		IF	ID	EXE	MEM	WB			
Instruction x+2			IF	ID	EXE	MEM	WB		
Instruction x+3				IF	ID	EXE	MEM	WB	
Instruction x+4					IF	ID	EXE	MEM	WB

Figure 7: Simple 5-stage pipeline diagram.

Pipelining

add x1, x2, x3

ID Two registers, x2 and x3, are read from the register file

WB The result is written into x1 in the register file

ld x1, offset(x2)

ID the x2 register value is read from the register file.

EXE The ALU computes the sum of X2 with the offset

MEM The result is used as the address for the data memory.

WB The data from the memory unit is written into the register file

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load doubleword (ld)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store doubleword (sd)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, and, or)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

FIGURE 4.24 Total time for each instruction calculated from the time for each component.

Pipelining

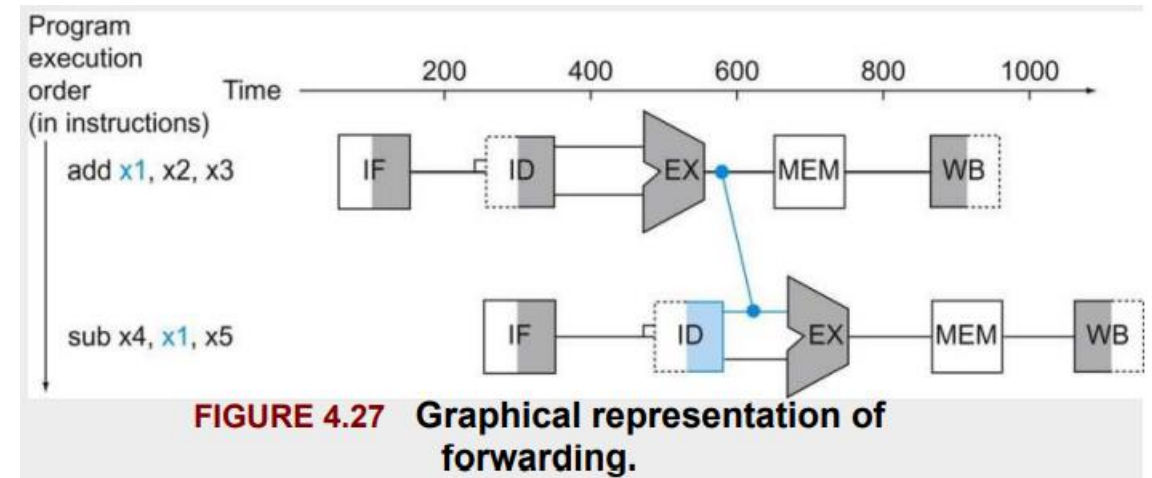
- Most modern CPUs are deeply pipelined, aka super pipelined
- Throughput: the number of instructions that complete and exit the pipeline per unit of time
- *In theory* the time required to move an instruction from one stage to the other can define the basic machine cycle or clock for the CPU.
 - Actually, some stages can require large execution times (e.g. a division can take 10 to 20 clock cycles), so faster cycles to speedup the most common cases (sum vs sqrt, cached data vs non cached ones)
- Hazards limit the performance.

Hazards

- **Structural** hazards are due to resource conflicts.
Hardware resources replication.
- **Control** hazards are caused due to the changes in the program flow.
Branch prediction and speculative execution.
- **Data** hazards are caused by data dependencies.
Bypassing/forwarding or
Out-of-order execution

Data Hazards

- It does not solve all the possible issues, but **mitigates** them.
- E.g. if the first instruction is LD x1 we'll have the value at the fourth stage.



- out-of-order (OOO) execution: sequential instructions can enter the execution pipeline stage in any **arbitrary order** only limited by their dependencies.
- BUT OOO execution CPUs must still give **the same result** as if all instructions were executed in the program order.
- An instruction is called **retired** when its results are correct and visible in the architectural state.

Instruction	Clock cycle									
	1	2	3	4	5	6	7	8	9	10
Instruction x	IF	ID	EXE	MEM	WB					
Instruction x+1		IF	ID			EXE	MEM	WB		
Instruction x+2			IF	ID	EXE	MEM			WB	
Instruction x+3				IF	ID		EXE	MEM		WB

Figure 8: The concept of Out-Of-Order execution.

Structural hazards

- Traditionally: one instruction at a time.
 - Now multiple Eus
- Superscalar CPUs: they can issue 2-6 instructions at a time
 - Scheduling via hardware or compiler with the Very Long Instruction Word

Instruction	Clock cycle					
	1	2	3	4	5	6
Instruction x	IF	ID	EXE	MEM	WB	
Instruction x+1	IF	ID	EXE	MEM	WB	
Instruction x+2		IF	ID	EXE	MEM	WB
Instruction x+3		IF	ID	EXE	MEM	WB

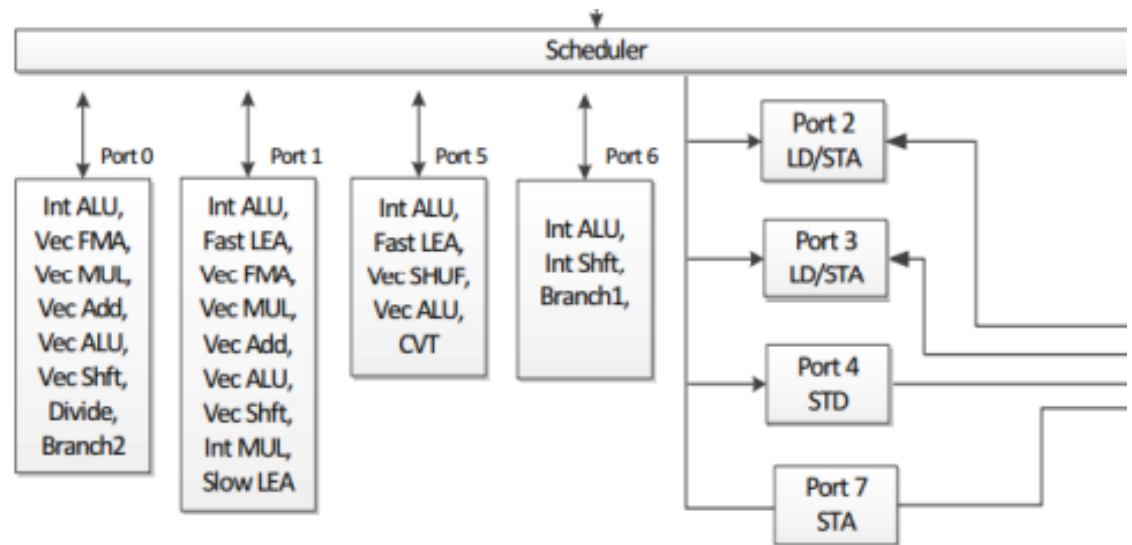


Figure 9: The pipeline diagram for a simple 2-way superscalar CPU.

Intel skylane

Branch Prediction

- Branches are points in the instruction stream where the execution may jump to another location, instead of executing the next instruction.
- Processors, instead of waiting for the loop to branch route outcome, try to **predict** the action that will be taken, in order to keep the pipeline full
- First possibility: at the bottom of loops are branches that jump back to the top of the loop. Since they are likely to be taken and they branch backward, we could always predict taken for branches that jump to an earlier address.
- Otherwise, dynamic hardware predictors make their guesses depending on the behaviour of each branch and may change predictions for a branch over the life of a program
- One popular approach to dynamic prediction of branches is *keeping a history* for each branch as taken or untaken, and then using the recent past behaviour to predict the future. The amount and type of history kept have become extensive, with the result being that dynamic branch predictors can correctly predict branches with more than 90% accuracy.
- In every case when the guess is wrong, the pipeline control must ensure that the instructions following the wrongly guessed branch have no effect **and must restart the pipeline from the proper branch address**. Longer pipelines exacerbate the problem, in this case by raising the cost of misprediction

Speculative execution

- The CPU takes a guess on an outcome of the branch and initiates processing instructions from the chosen path
- State changes to the machine cannot be committed until the condition is resolved to ensure that the architecture state of the machine is never impacted by speculatively executing instructions.
 - A branch instruction can be dependent on a value loaded from memory, which can take hundreds of cycles

Listing 3 Speculative execution

```
if (a < b)
    foo();
else
    bar();
```

Instruction	Clock cycle							
	1	2	3	4	5	6	7	8
BRANCH (a < b)	IF	ID	EXE	MEM	WB			
CALL foo				IF	ID	EXE	MEM	WB
// INSTR from foo					IF	ID	EXE	MEM

(a) No speculation

Instruction	Clock cycle						
	1	2	3	4	5	6	7
BRANCH (a < b)	IF	ID	EXE	MEM	WB		
CALL foo		IF*	ID*	EXE	MEM	WB	
// INSTR from foo			IF*	ID	EXE	MEM	WB

(b) Speculative execution

To summarize

- A pipelined processor can speed up operations by a factor of 4 (or more) with respect to earlier CPUs.
- But dependencies and branches have to be taken into account
- These days, CPUs can have 20-stage pipelines or more (up to 31).

Are they incredibly fast?

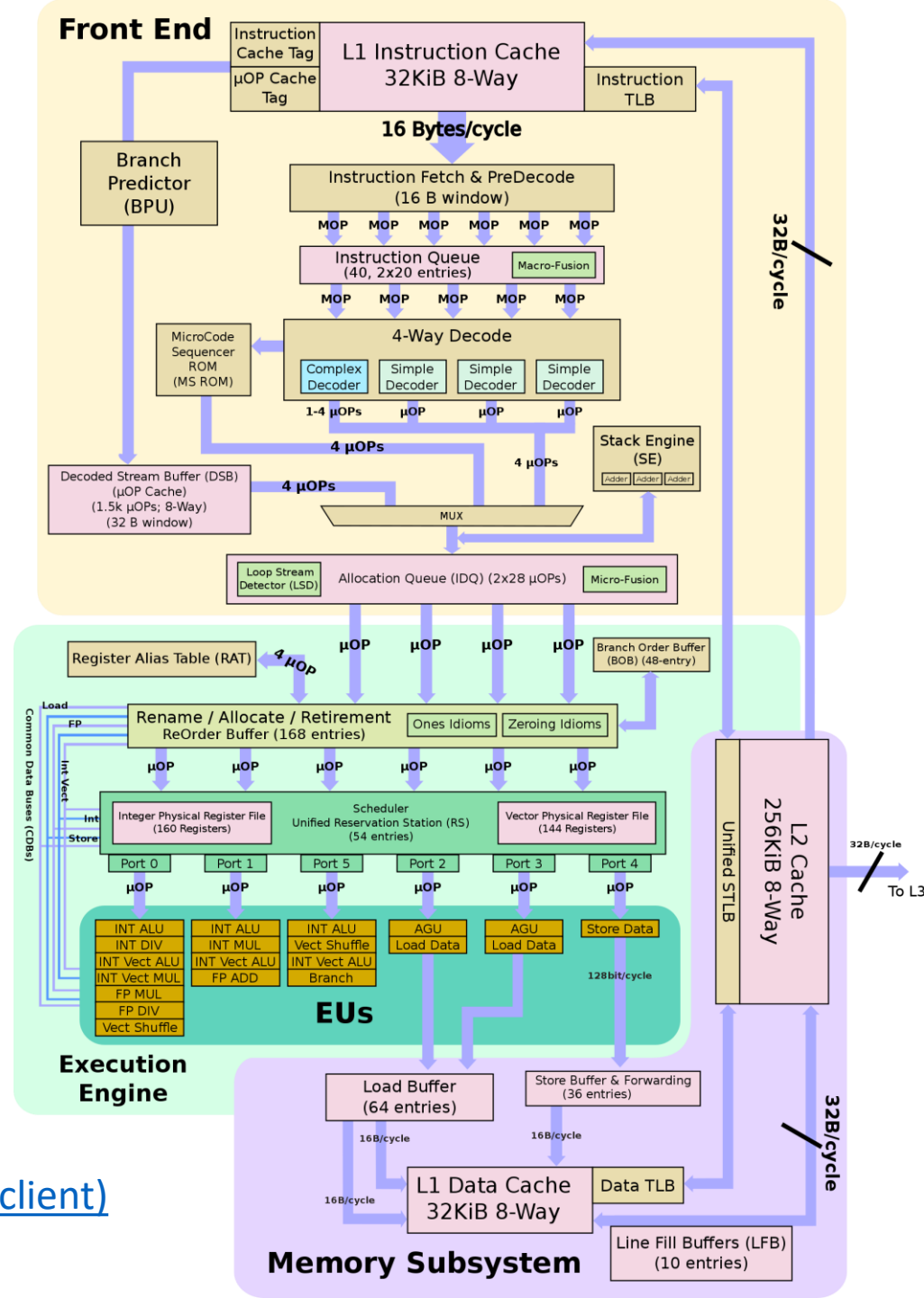
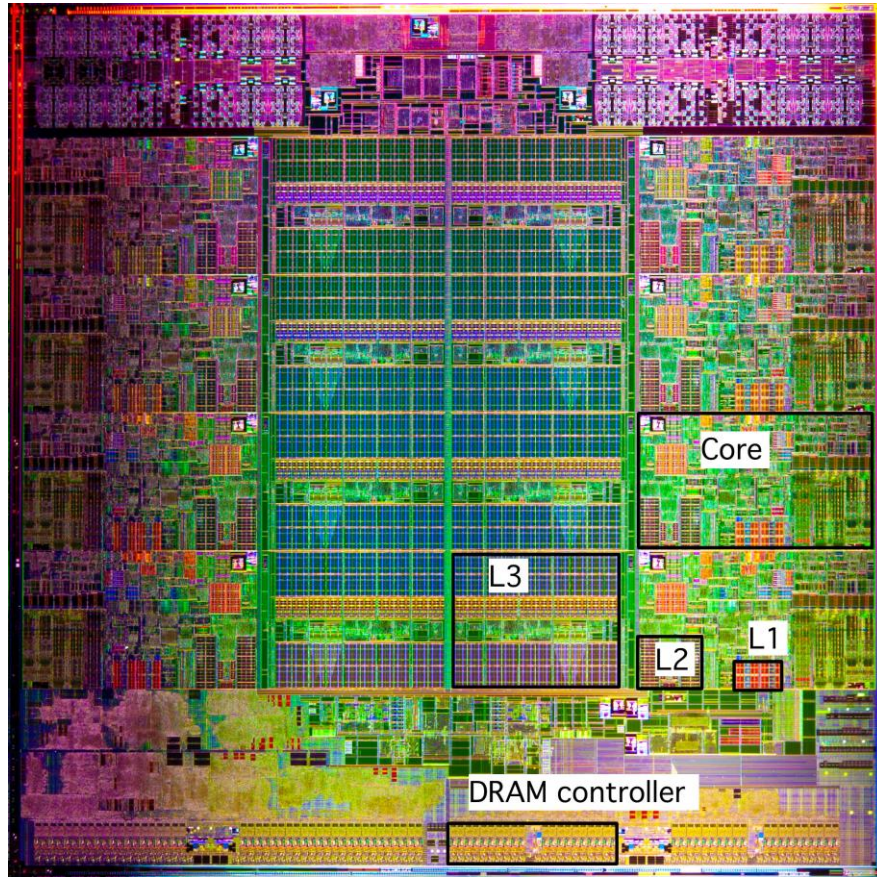
<https://www.hardwaresecrets.com/inside-pentium-4-architecture/2/>

<https://www.gamedev.net/tutorials/programming/general-and-gameplay-programming/a-journey-through-the-cpu-pipeline-r3115/>

- Chip designers continue to increase the clock rate, and the pipeline segments can no longer finish their work in one cycle, so they are further split up. Sometimes there are even segments in which nothing happens: that time is needed to make sure data can travel to a different part of the chip in time.
- Thanks to pipelining, for modern CPUs there was a simple (maybe too simple) relation between the clock speed and the **peak** performance: #FPU x clock speed
- BUT now vector operations and multicore...

What does a CPU look like?

The Intel Sandy Bridge processor

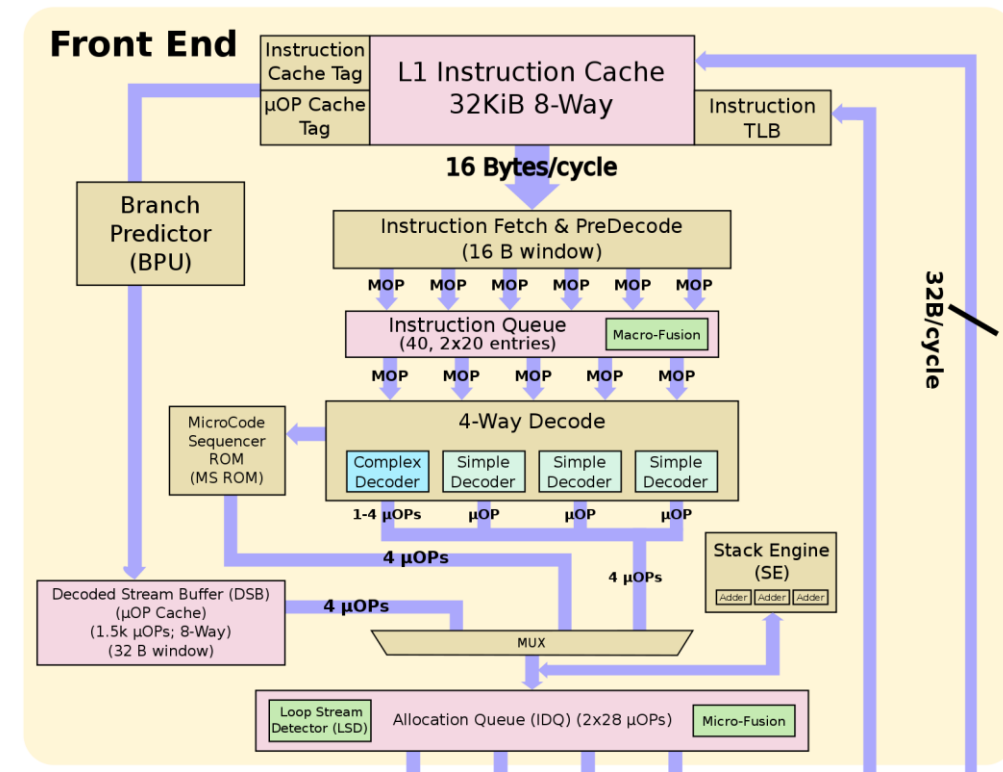


[https://en.wikichip.org/wiki/intel/microarchitectures/sandy_bridge_\(client\)](https://en.wikichip.org/wiki/intel/microarchitectures/sandy_bridge_(client))

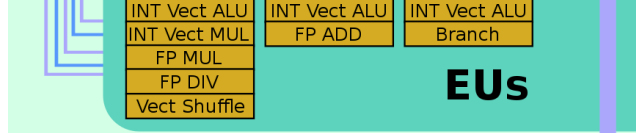
<https://www.realworldtech.com/sandy-bridge/3/>

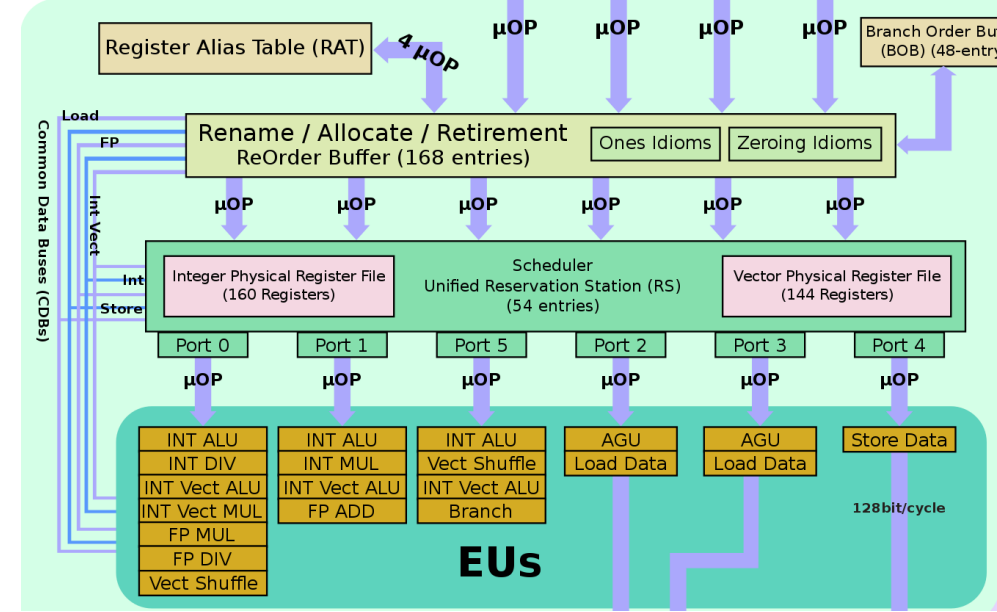
Sandy Bridge – Front End

- The **front-end** is tasked with the challenge of fetching the complex x86 instructions from memory, decoding them, and delivering them to the execution units.
- x86 instructions are complex and variable length. At the **pre-decode** buffer the instructions boundaries get detected and marked. The result is a set of Macro-Operation (MOP), a more complex version of a micro-operation as handled by the microprocessor.
- The **branch prediction unit** (BPU) attempts to guess the flow of instructions. Due to the deep pipeline, a flush is a rather expensive event which ends up discarding over 150 instructions that are in-flight.
- The pre-decoded instructions are delivered to the **Instruction Queue**, that does macro-op fusion if possible, e.g. single compare-and-branch instructions.



Sandy Bridge – Execution Engine

- It deals with the execution of out-of-order operations and treats the three classes of μ OPs (Floating Point, INTeger and Vector) separately.
 - On each cycle, up to 4 μ OPs can be delivered here from the front-end. At this stage architectural registers are mapped onto the underlying physical registers. Other additional bookkeeping tasks are also done at this point such as allocating resources for stores, loads, and determining all possible scheduler ports.
 - The scheduler holds the μ OPs while they wait to be executed. A μ OP could be waiting on an operand that has not arrived (e.g., fetched from memory or currently being calculated from another μ OPs) or because the execution unit it needs is busy. Once the μ OP is ready, they are dispatched through their designated port.
 - The scheduler will send the oldest ready μ OP to be executed on each of the six ports each cycle. Port 0, 1, and 5 are used for executing computational operations. Ports 2, 3, and 4, are used for executing memory related operations such as loads and stores. Sandy Bridge introduced the AVX extension, a new 256-bit x86 floating point SIMD extension.
 - Once a μ OP executes it can be retired.
- 



ILP

- In the von Neumann model processors operate through **control flow**: instructions follow each other linearly or with branches without regard for what data they involve.
- Today CPUs switched to the **data flow** model: they analyze several instructions to find data dependencies, and execute instructions in parallel that do not depend on each other
 - instructions that are independent can be started at the same time
 - Independent instructions can be rearranged if the resulting execution will be more efficient (out-of-order execution)
 - data can be speculatively requested before any instruction needing it is actually encountered (prefetching)
 - Pipelining and branch prediction

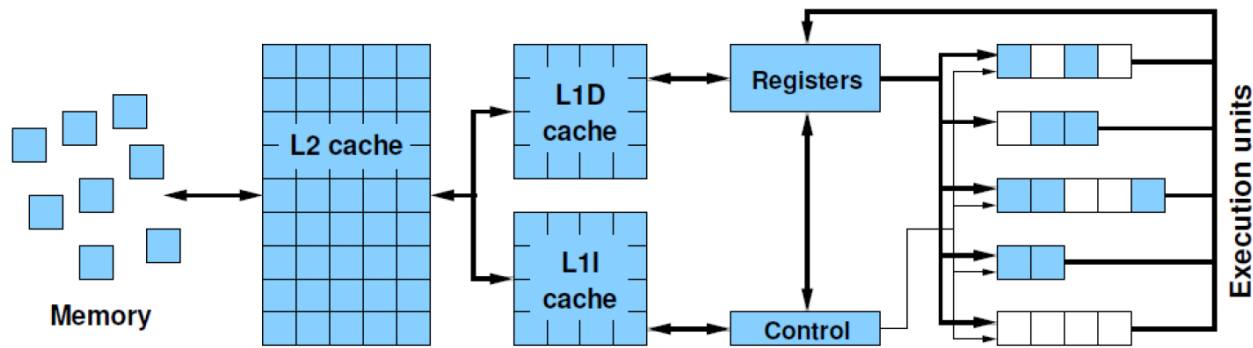
From ILP to multicore

- As clock frequency has gone up, the processor pipeline has grown in length to make the segments executable in less time.
- However ILP has its limits, therefore making pipelines longer (sometimes called 'deeper') no longer pays off
 - E.g. branch misprediction results in branch penalty
- Therefore chip designers have moved to multicore architectures as a way of more efficiently using the transistors on a chip

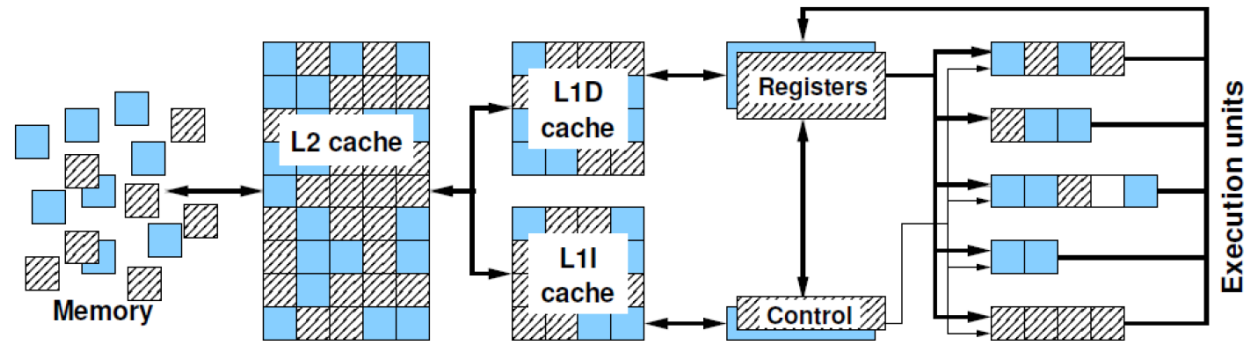
Simultaneous MultiThreading

SMT makes a **single physical core** appear as two or more “logical” cores, i.e. multiple threads/processes run concurrently. It can increase the throughput.

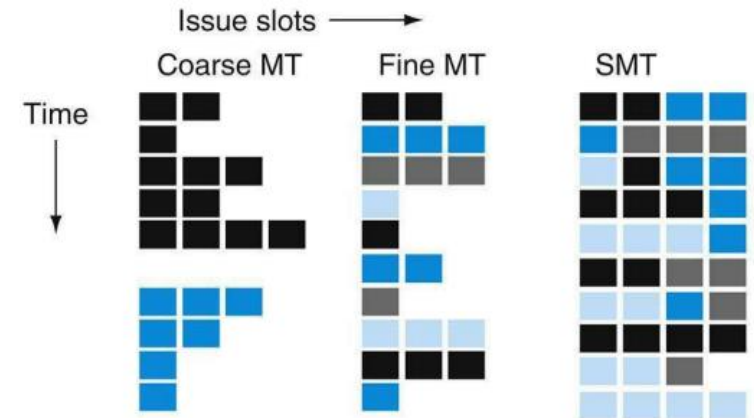
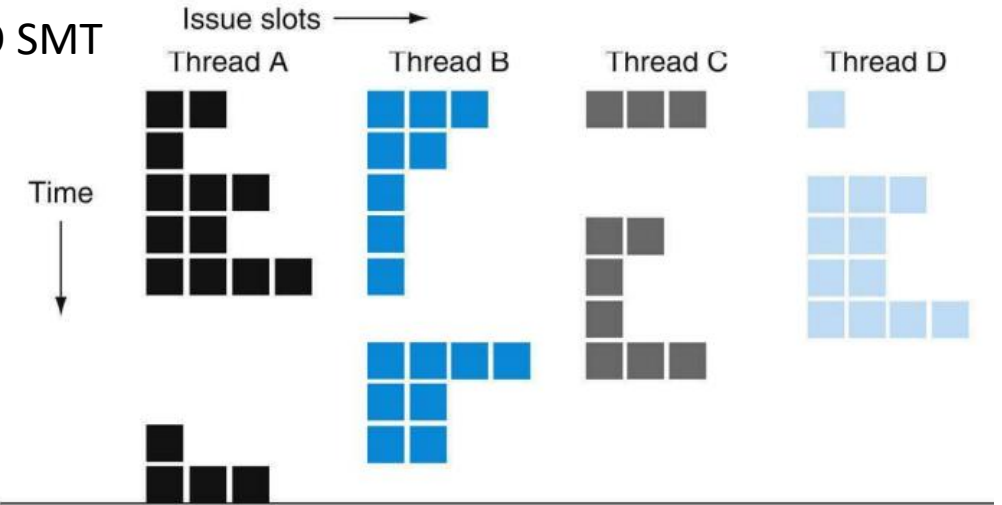
Standard core



2-way SMT



NO SMT



SMT

- Possible benefit: **Better pipeline throughput**
 - Filling otherwise unused pipelines
 - Filling pipeline bubbles with other thread's executing instructions:

Thread 0:

```
do i=1,N
  a(i) = a(i-1)*c
enddo
```

Dependency → pipeline
stalls until previous MULT
is over

Thread 1:

```
do i=1,N
  b(i) = s*b(i-2)+d
enddo
```

Unrelated work in other
thread can fill the pipeline
bubbles

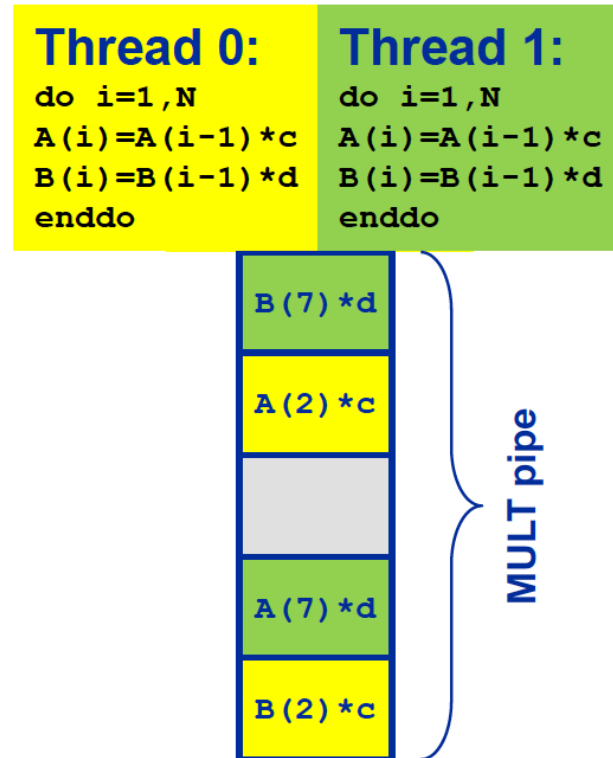
- **Beware:** Executing it all in a single thread
(if possible) may reach the same goal
without SMT:

```
do i=1,N
  a(i) = a(i-1)*c
  b(i) = s*b(i-2)+d
enddo
```

The ideal workload for SMT

Simple loop-carried dependency benchmark $A(i) = s * A(i-1)$

- Bottleneck: MULT pipeline latency
 - Haswell CPU: 5 cy/it best case
- Running 2 threads via SMT: expect 2.5 cy/it if no other bottlenecks turn up
- Further improvement?
 - Multiple independent streams of instructions per thread
- What about the data transfer?



Many more details:

<http://joomla.di.unipi.it/~vannesch//SPA/p29-ungerer.pdf>

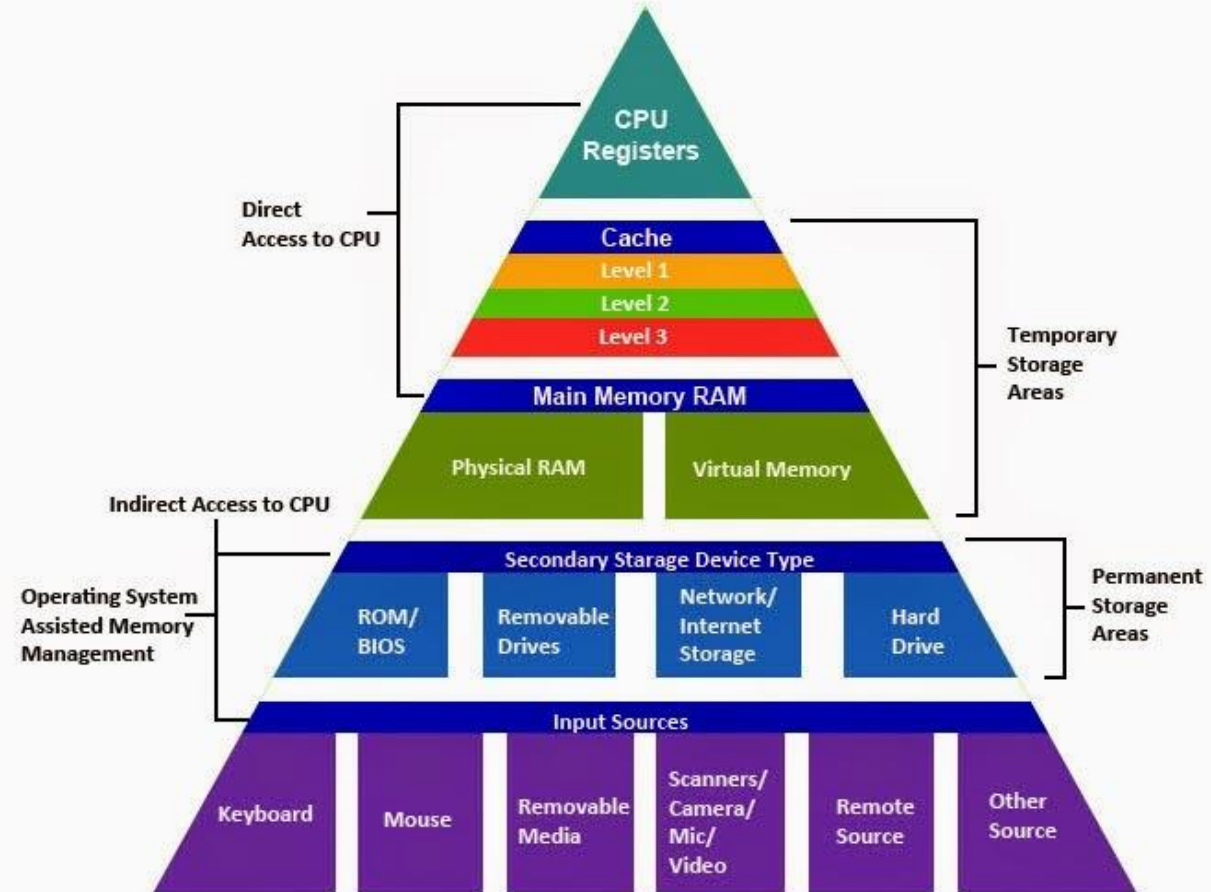
To summarize

- Allows the CPU to switch to another task when the current task is stalled
 - A *context switch* has essentially zero cost
 - The CPU can switch to another task even on stalls of short durations (e.g., waiting for memory operations), so it runs >1 instruction stream (thread/process) at a time
 - Requires CPU with specific support
 - Full register set available per SMT thread
- If the bottleneck is not core execution, SMT will be of limited use
- HyperThreading is Intel's implementation of SMT
- Use `lscpu` / `lstopo` in Linux to get details

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Byte Order:            Little Endian
CPU(s):                256
On-line CPU(s) list:   0-255
Thread(s) per core:    4
Core(s) per socket:    64
Socket(s):             1
NUMA node(s):          1
Vendor ID:             GenuineIntel
CPU family:            6
Model:                87
Model name:            Intel(R) Xeon Phi(TM) CPU 7210 @ 1.30GHz
Stepping:              1
CPU MHz:               1501.601
BogoMIPS:              2600.12
L1d cache:             32K
L1i cache:            32K
L2 cache:              1024K
NUMA node0 CPU(s):    0-255
```

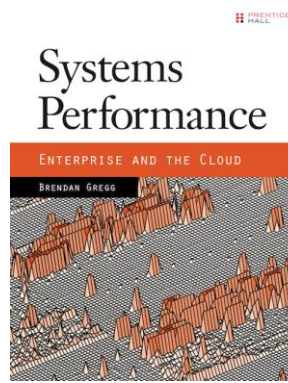
Memory hierarchy

- Von Neuman architecture: data is loaded immediately from memory to the processors, where it is operated on
- There are several memory levels, the memory hierarchy. These try to alleviate **the memory wall problem** by making **recently used data available quicker** than it would be from main memory.
- Of course, this presupposes that the algorithm and its implementation allow for data to be used multiple times.



Memory Hierarchy

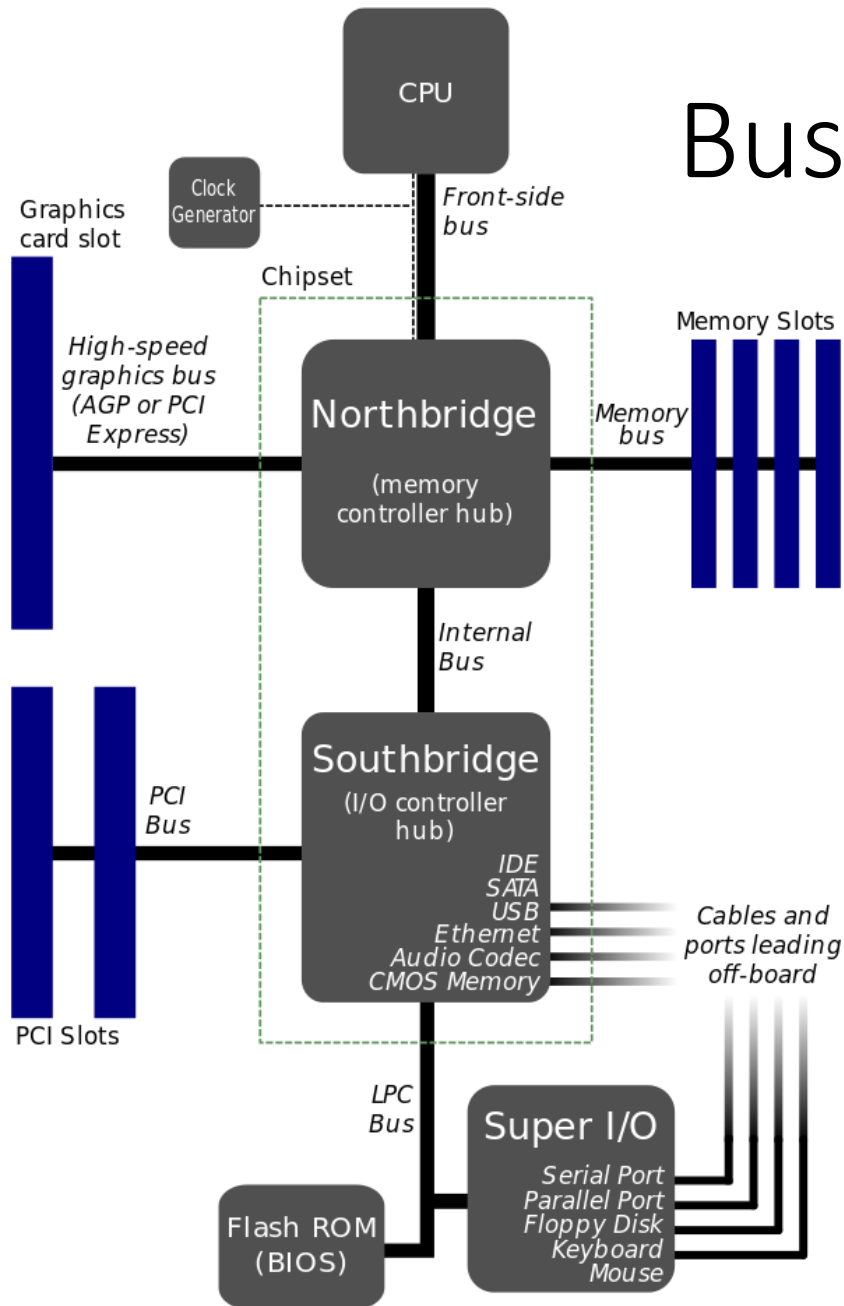
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access	120 ns	6 min
Solid-state disk I/O	50-150 μ s	2-6 days
Rotational disk I/O	1-10 ms	1-12 months
Internet: SF to NYC	40 ms	4 years
Internet: SF to UK	81 ms	8 years
Internet: SF to Australia	183 ms	19 years
OS virtualization reboot	4 s	423 years
SCSI command time-out	30 s	3000 years
Hardware virtualization reboot	40 s	4000 years
Physical system reboot	5 m	32 millenia



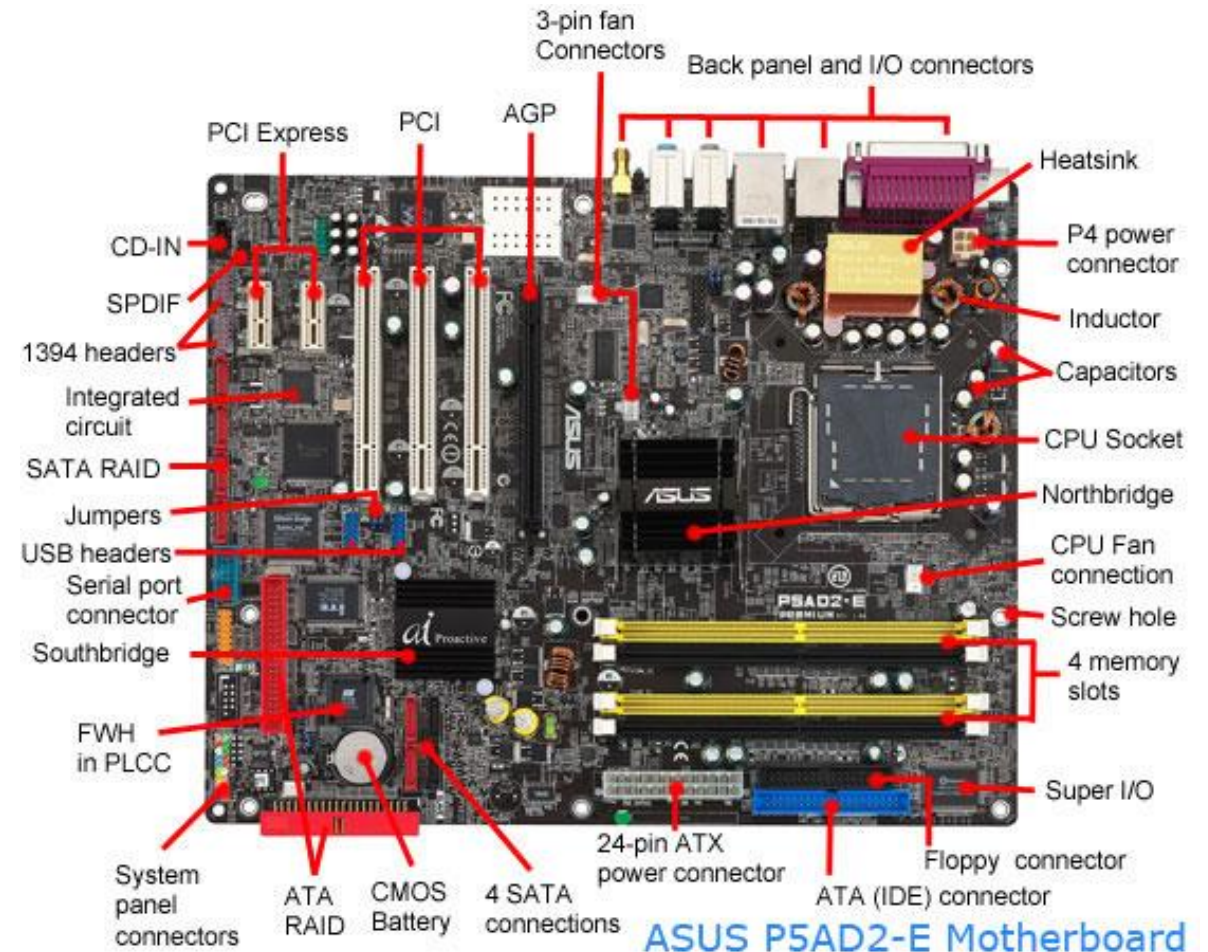
- Clock Speed
- Execution Optimization
- **Cache**

The **computer performance shell game**, also known as "find the bottleneck", is played between four resources: CPU, Disk, Network, Memory. But which one? How long will you wait?

Buses



- The wires that move data around in a computer are called buses.
- The bus is typically slower than the processor.
- The bandwidth of a bus is also determined by the number of bits that can be moved per clock cycle.



ASUS P5AD2-E Motherboard

Latency and Bandwidth

The two most important terms related to performance for memory subsystems and networks:

- **Latency**: is the delay between the processor issuing a request for a memory item, and the item actually arriving
- It is generally measured in nanoseconds (milliseconds for network latency) or clock periods.
- Sometimes addresses are predictable and compiler will schedule the fetch. Otherwise the out-of-order execution can hide this time (latency hiding)
- **Bandwidth**: the rate at which data arrives at its destination, after the initial latency is overcome
- Units are B/sec (MB/sec, GB/sec, etc.)
- The time needed to transfer n byte is

$$T(n) = \alpha + \beta n$$

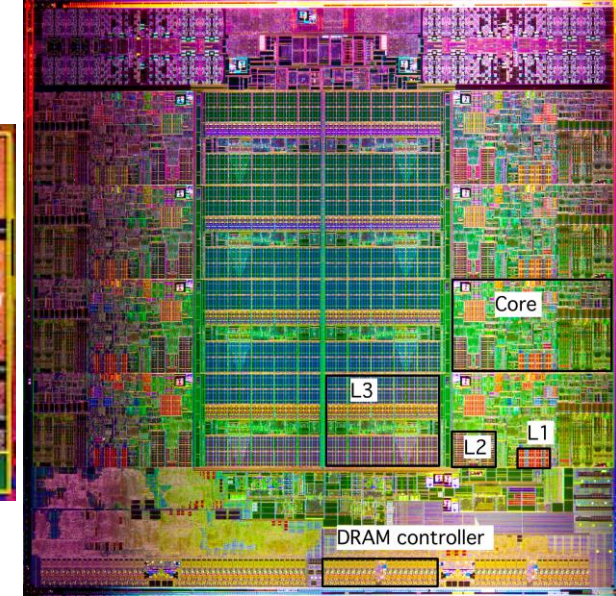
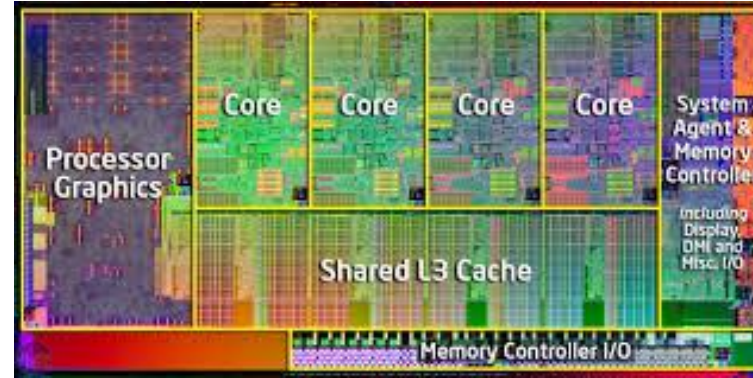
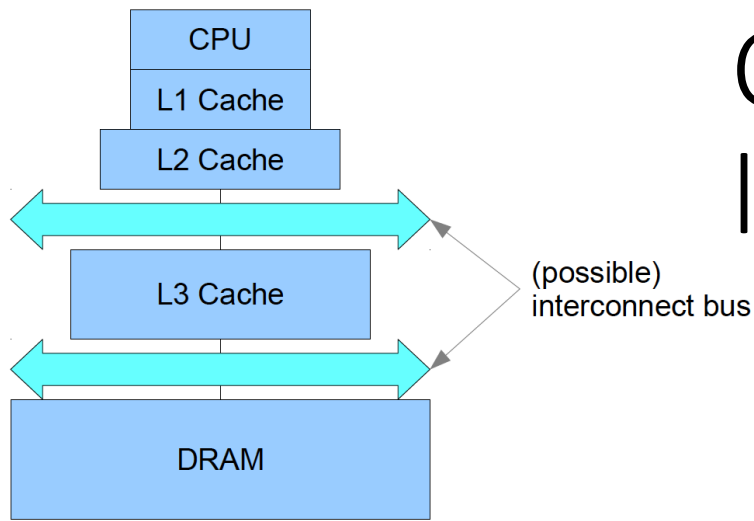
α = latency, a constant value

β = 1/bandwidth, the time per byte

Caches

- In between the registers, which contain the immediate input and output data for instructions, and the main memory where lots of data can reside for a long time, are various levels of cache memory, that have lower latency and higher bandwidth than main memory and where data are kept for an intermediate amount of time.
- Data from memory travels through the caches to wind up in registers. The advantage to having cache memory is that if a data item is reused shortly after it was first needed, it will still be in cache, and therefore it can be accessed much faster than if it would have to be brought in from memory.
- There is an important difference between cache memory and registers: while data is moved into register by explicit assembly instructions, the move from main memory to cache is entirely done by hardware.
- Thus cache use and reuse **is outside of direct programmer control**. You can only **influence it** by exploiting data locality.

Cache levels



- L1 Cache: Data cache closest to registers
 - It is usually split into instruction cache and the data cache
- L2 Cache: Secondary data cache, stores both data and instructions
 - Data from L2 has to go through L1 to registers
 - L2 is 10 to 100 times larger than L1
- Most systems have an L3 cache, ~10x larger than L2
- Cache line
 - The smallest unit of data transferred between main memory and the caches (or between levels of cache)
 - N sequentially-stored, multi-byte words. Typically 64 **bytes** (i.e. 8 double values).

Details

On my 2 machines

Install perf and

<https://github.com/RRZE-HPC/likwid/wiki>

/usr/local/bin/likwid-topology -g

Socket 0:

+-----+ +-----+ +-----+ +-----+			
0	1	2	3
+-----+ +-----+ +-----+ +-----+			
32 kB	32 kB	32 kB	32 kB
+-----+ +-----+ +-----+ +-----+			
256 kB	256 kB	256 kB	256 kB
+-----+ +-----+ +-----+ +-----+			
6 MB			
+-----+ +-----+ +-----+ +-----+			

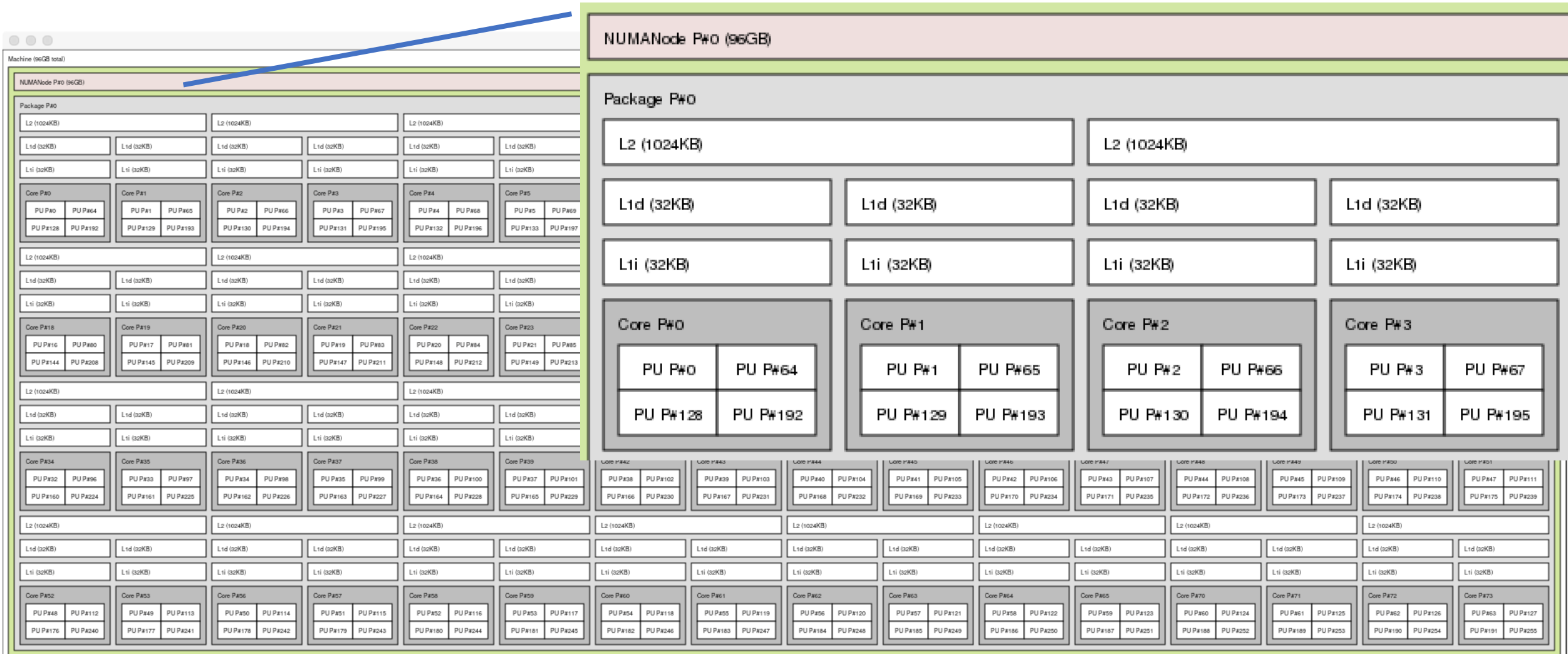
Socket 0:

+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
0 12	1 13	2 14	3 15	4 16	5 17
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
32 kB	32 kB	32 kB	32 kB	32 kB	32 kB
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
256 kB	256 kB	256 kB	256 kB	256 kB	256 kB
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
12 MB					
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					

Socket 1:

+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
6 18	7 19	8 20	9 21	10 22	11 23
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
32 kB	32 kB	32 kB	32 kB	32 kB	32 kB
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
256 kB	256 kB	256 kB	256 kB	256 kB	256 kB
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					
12 MB					
+-----+ +-----+ +-----+ +-----+ +-----+ +-----+					

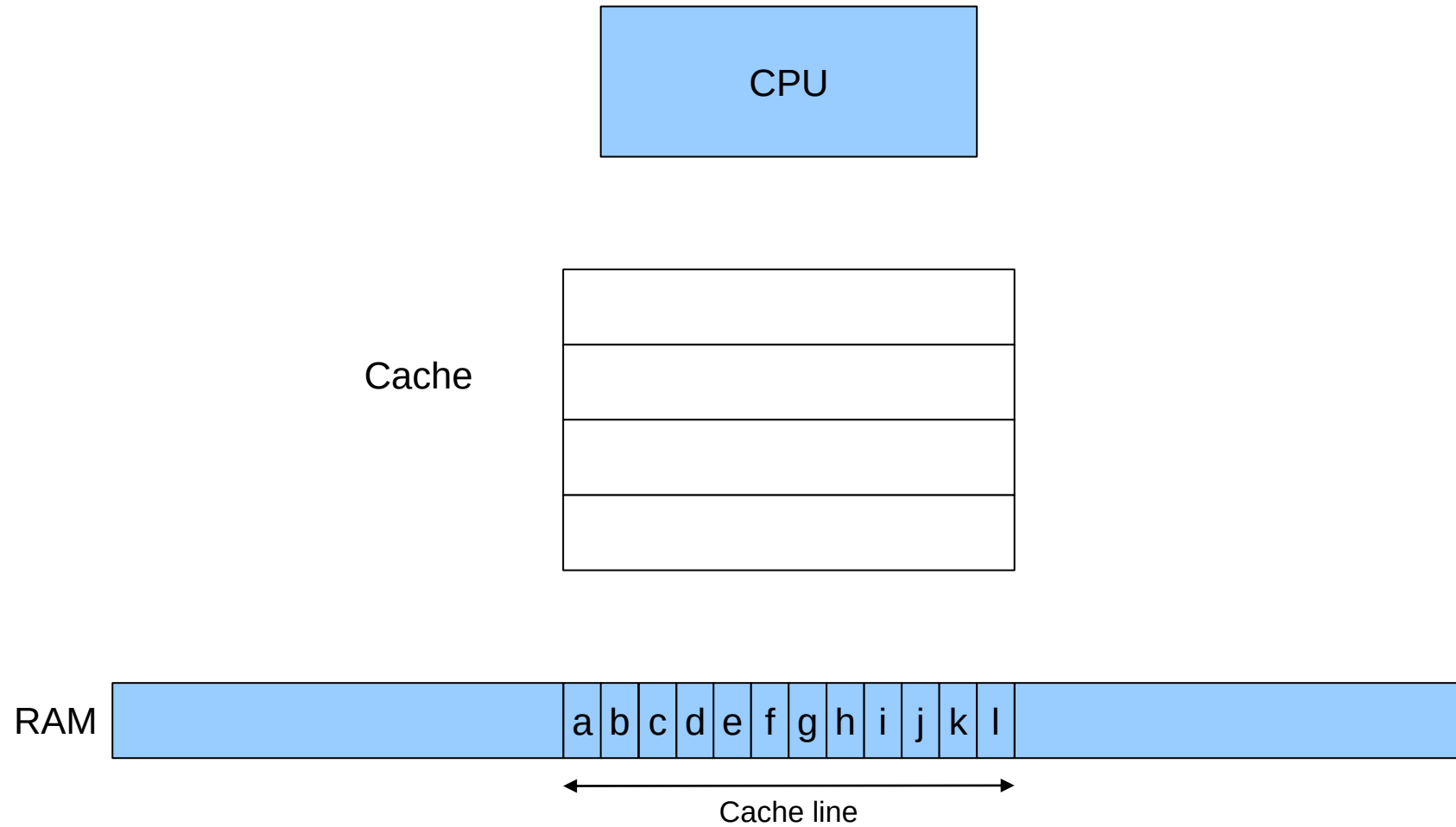
Details – KNL Xeon Phi 7210 (64 cores * 4 thread)



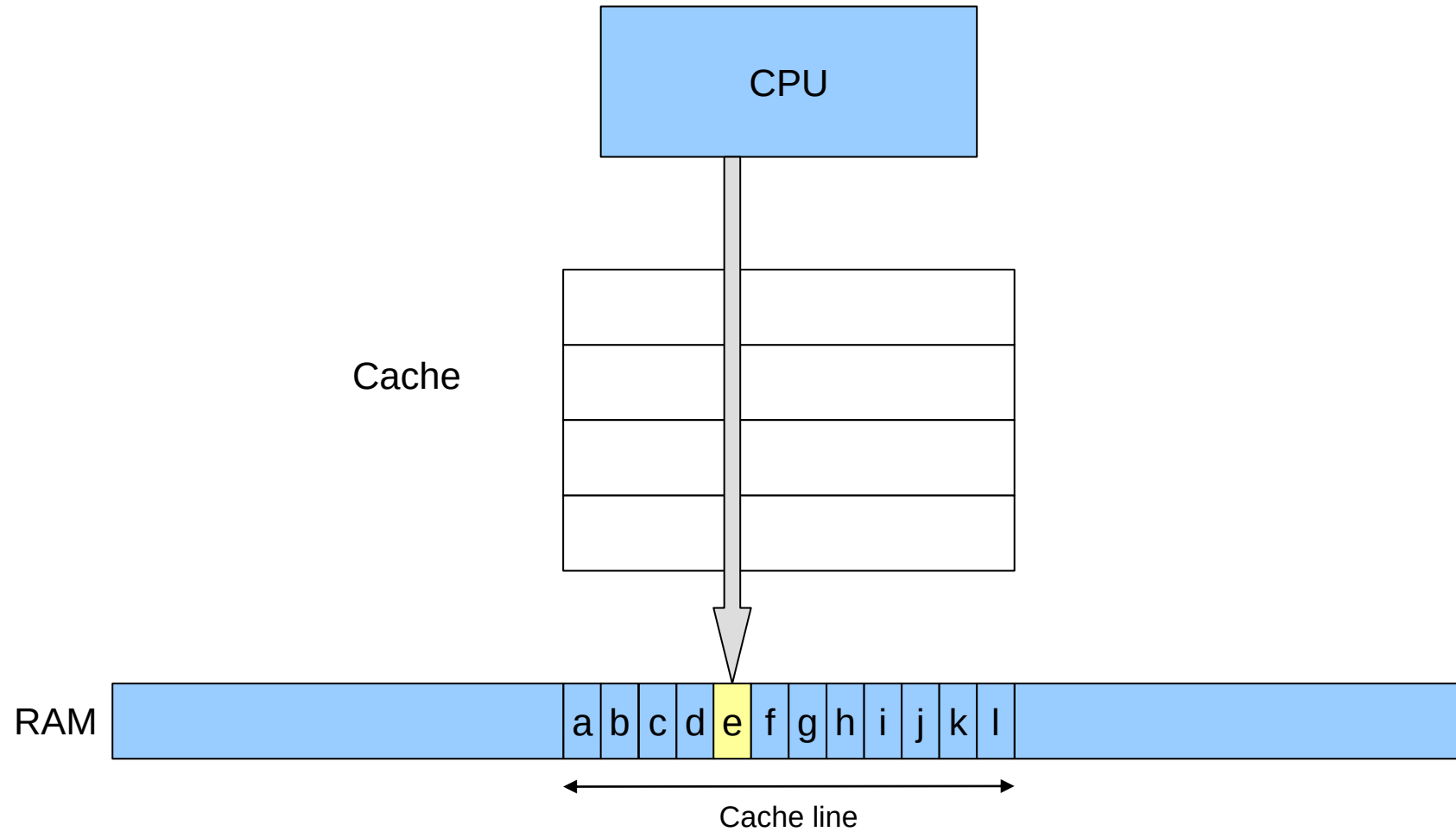
How the cache works

- Data needed in some operation gets copied into the various caches on its way to the processor.
- If, some instructions later, a data item is needed again, it is first searched for in the L1 cache; if it is not found there, it is searched for in the L2 cache; if it is not found there, it is loaded from main memory.
- Finding data in cache is called a cache hit, and not finding it a cache miss.
- Unlike main memory, which is expandable, caches are fixed in size. If a version of a processor chip exists with a larger cache, it is usually considerably more expensive.

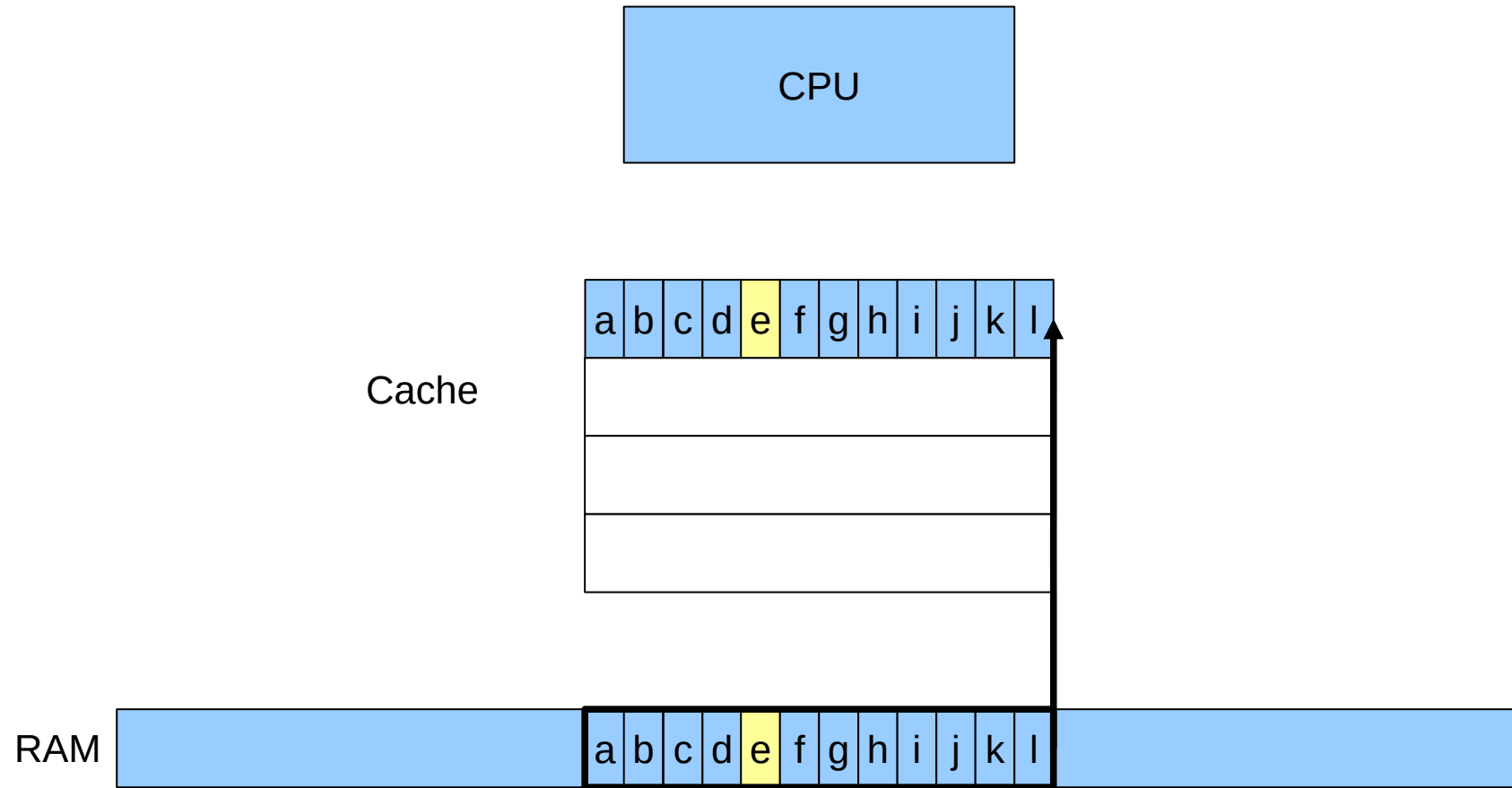
Example



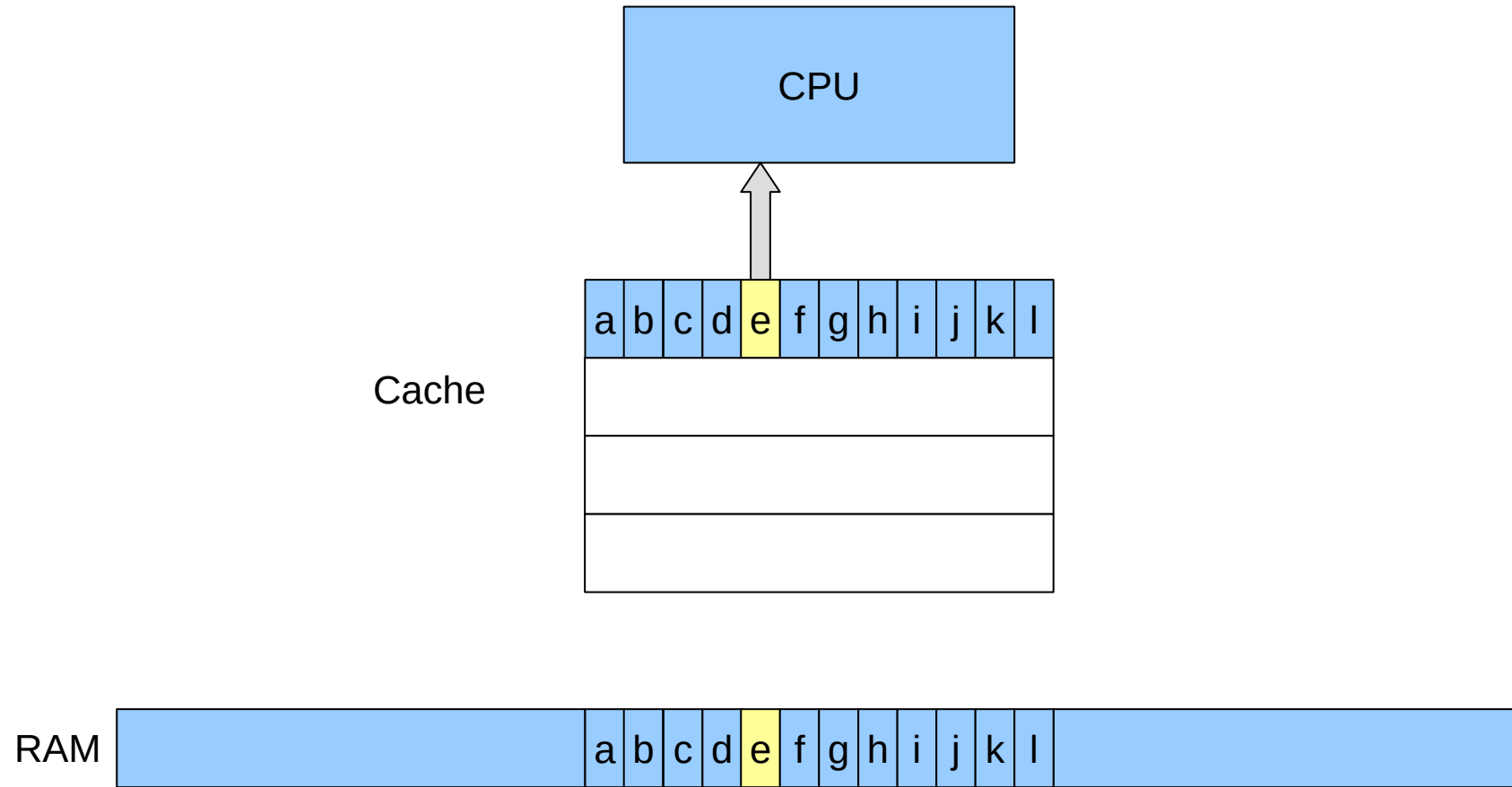
Example



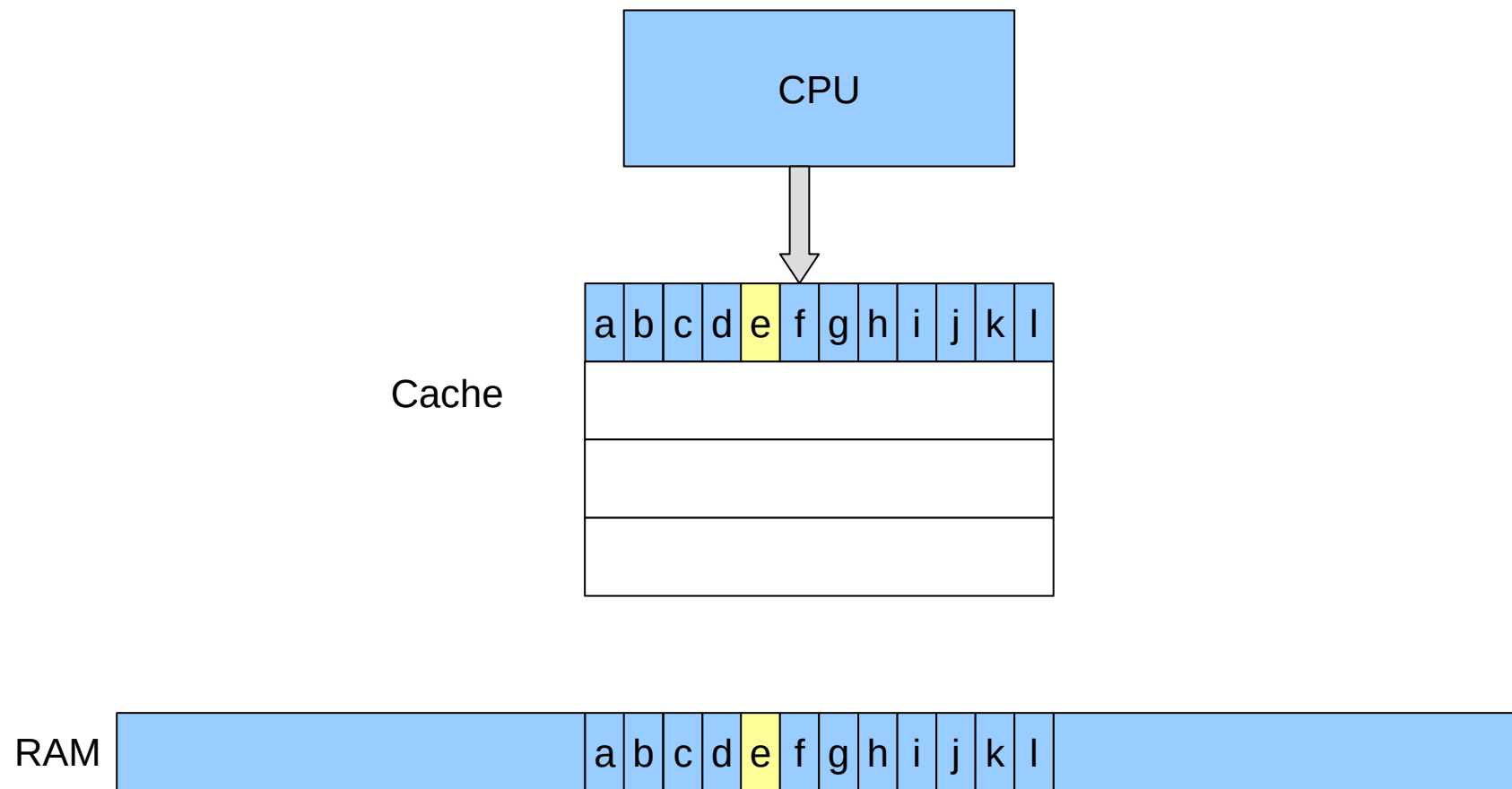
Example



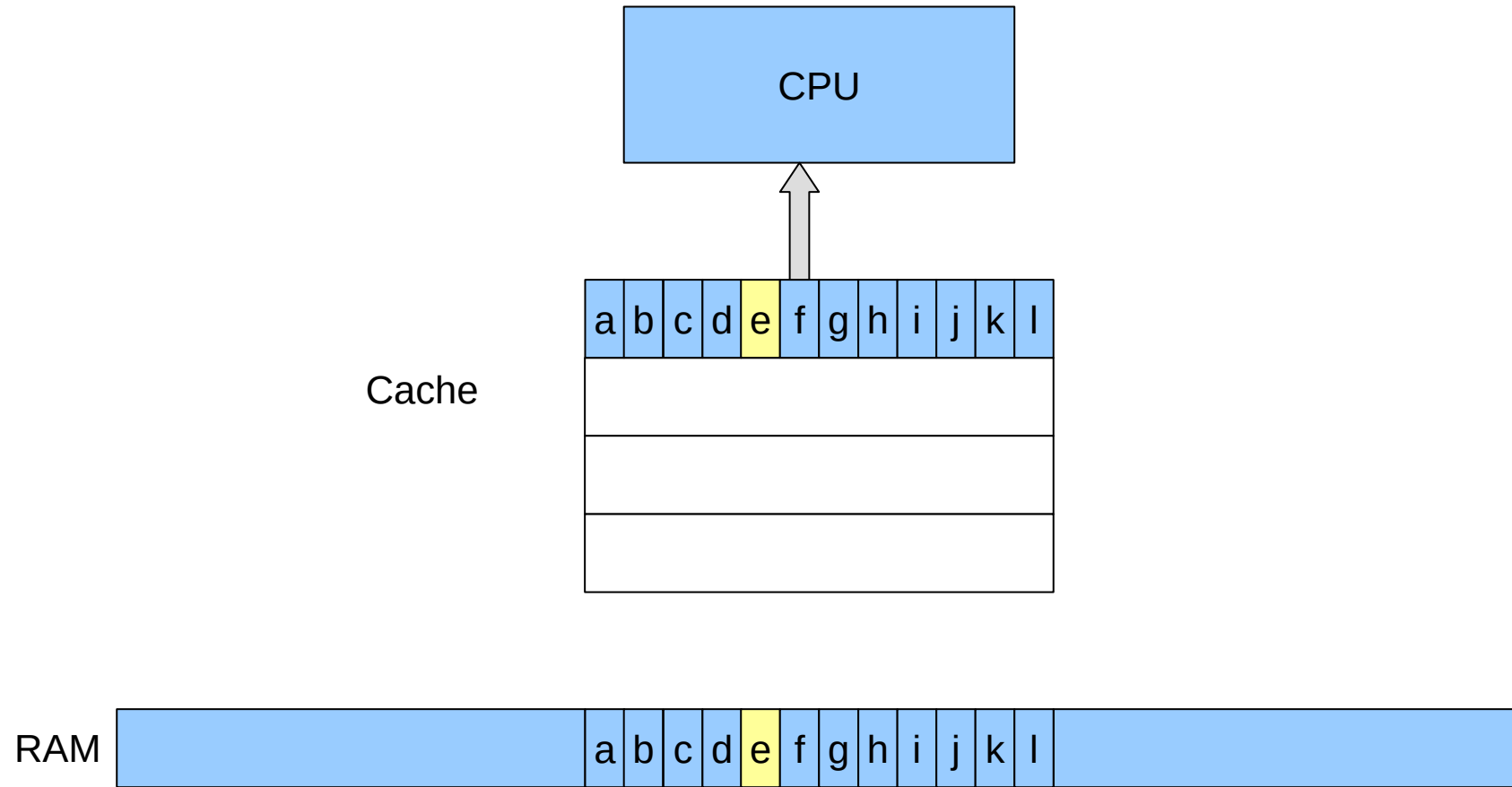
Example



Example



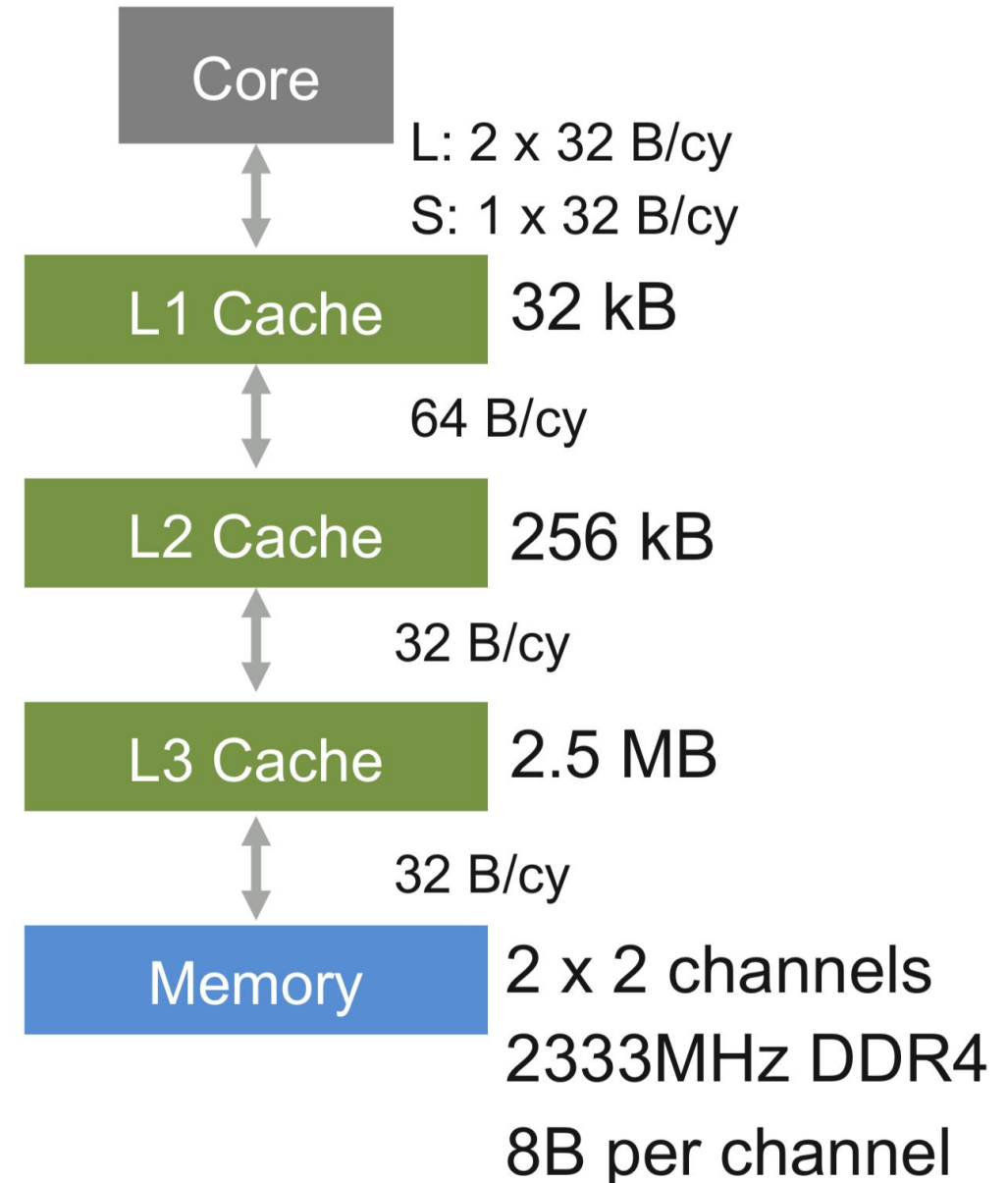
Example



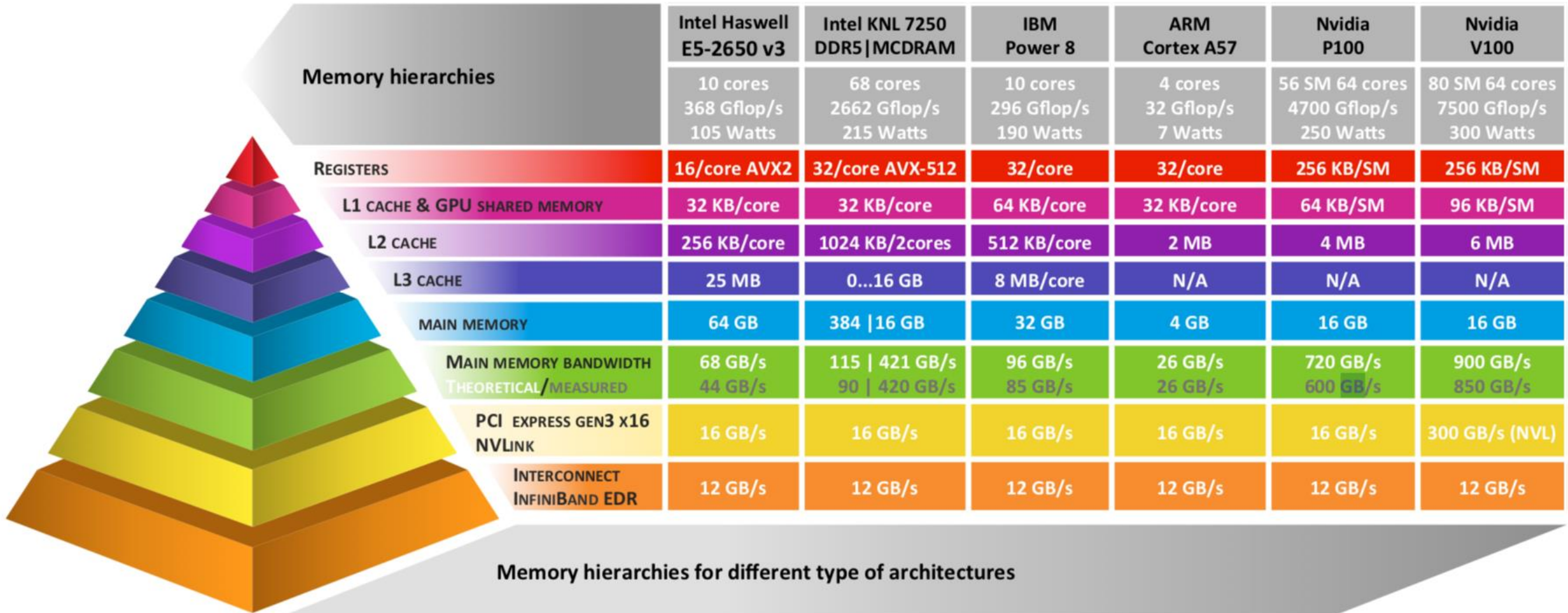
Terminology

- Cacheline (CL): Smallest unit of data that can be transferred within the memory hierarchy. On x86 usually 64 **Bytes**.
- Cacheline states
 - Modified (dirty): CL is only in current cache and does not match main memory
 - Exclusive (clean): CL is only in current cache and matches main memory
 - Shared (clean): CL is unmodified and may be stored in other caches
 - Invalid: CL is unused
- Cacheline evict: A cache miss triggers a CL allocation. The replaced CL is evicted from the cache.

Haswell @ 2.3GHz



Some numbers

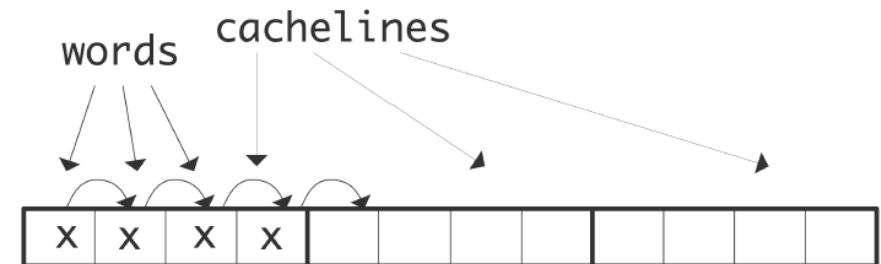


Cache line and stride

- An efficient program tries to use the other items on the cache line, since access to them is effectively free.
- This phenomenon is visible in code that accesses arrays by **stride**: elements are read or written at regular intervals.

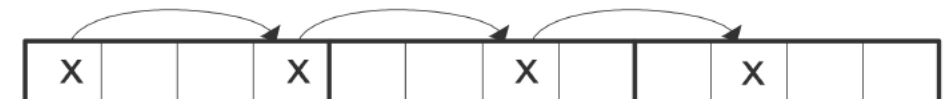
Perfect
use

```
for (i=0; i<N; i++)  
    ... = ... x[i] ...
```



Cache here
is useless

```
for (i=0; i<N; i+=stride)  
    ... = ... x[i] ...
```



Locality and Cache misses

Cache memory works well when applications exhibit spatial and/or temporal locality

- Spatial locality: Accessing adjacent memory locations is OK
- Temporal locality: Repeatedly accessing the same memory location(s) is OK

But, obviously, the first time you reference data you will always incur a cache miss

- This is called **compulsory cache miss**
- The second type of cache misses is due to the size of your working set: a **capacity cache miss** is caused by data having been overwritten because the cache can simply not contain all your problem data.
- The third type is **conflict misses**, caused by one data item being mapped to the same cache location as another, while both are still needed for the computation, and there would have been better candidates to evict.
- In a multicore system there is a fourth type, the invalidation miss. This happens if an item in cache has become invalid because another core changed the value of the corresponding memory address.

Temporal Locality

- This describes the use of a data element within a short time of its last use.
- Since most caches have an LRU replacement policy, if in between the two references less data has been referenced than the cache size, the element will still be in cache and therefore be quickly accessible.
- With other replacement policies, such as random replacement, this guarantee can not be made.
- Which example is likely to provide higher performance?

```
for (loop=0; loop<10; loop++) {  
    for (i=0; i<N; i++) {  
        ... = ... x[i] ...  
    }  
}
```

Subsequent use of x[i] are spaced too far for large N

```
for (i=0; i<N; i++) {  
    for (loop=0; loop<10; loop++) {  
        ... = ... x[i] ...  
    }  
}
```

Memory accesses are reduced to 1/10

Details (just a sketch)

Cache details in

/sys/devices/system/cpu/cpu0/cache/

(one dir per cache level)

Line of 64 bytes

perf stat -e task-clock,cycles,instructions, cache-references, cache-misses,branches,faults ./<executable>

```
#include <stdio.h>
#include <stdlib.h>

#define N 10000000

int main() {
    float* x;
    int i, loop;
    x = (float*) malloc(N*sizeof(float));

    for(i=0; i<N; ++i) x[i]=1.0;
    for(i=0; i<N; ++i) {
        for (loop = 0; loop < 10; loop++) {
            x[i] = x[i]+1.0;
        }
    }
    return(0);
}
```

Cycle on N followed by loop

248,317799	task-clock	#	0,998 CPUs utilized
922294038	cycles	#	3,714 GHz
1663682890	instructions	#	1,80 insns per cycle
781000	cache-references	#	3,145 M/sec
600226	cache-misses	#	76,854 % of all cache refs
140667844	branches	#	566,483 M/sec
678	faults	#	0,003 M/sec

0,248922993 seconds time elapsed

Cycle on loop followed by N

278,346717	task-clock	#	0,997 CPUs utilized
1034468468	cycles	#	3,716 GHz
1594211469	instructions	#	1,54 insns per cycle
1195224	cache-references	#	4,294 M/sec
930542	cache-misses	#	77,855 % of all cache refs
110740474	branches	#	397,851 M/sec
678	faults	#	0,002 M/sec

0,279143032 seconds time elapsed

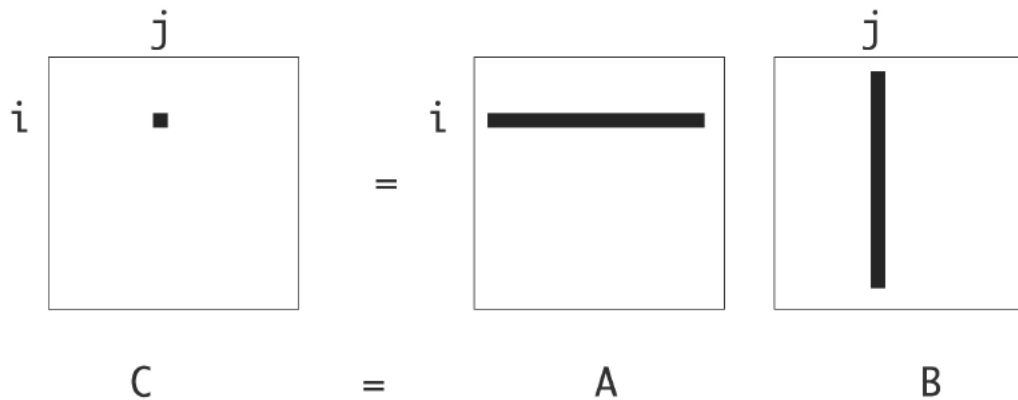
Details on the performance metrics

<https://www.intel.it/content/www/it/it/architecture-and-technology/64-ia-32-architectures-software-developer-system-programming-manual-325384.html>

Spatial Locality

Matrix-matrix product: $2n^3$ ops, $2n^2$ data (if we have $n \times n$ matrices)

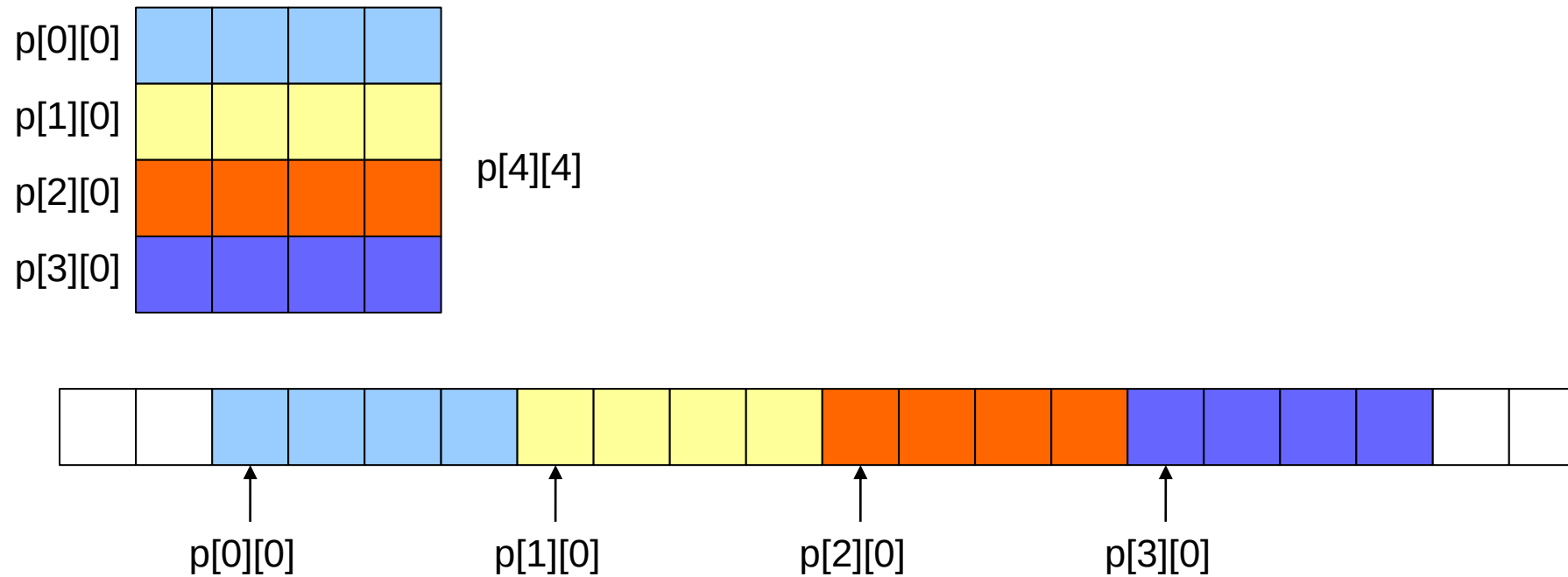
- Is it a cache-aware algorithm?



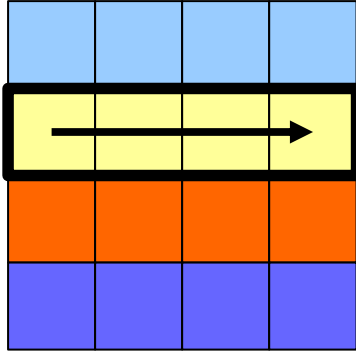
```
for i=1..n
  for j=1..n
    for k=1..n
      c[i,j] += a[i,k]*b[k,j]
```

Matrix representation

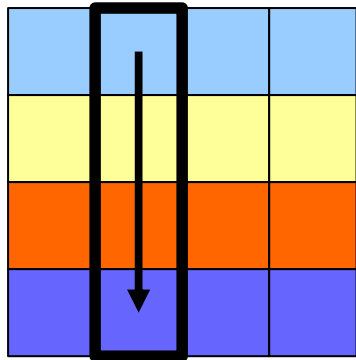
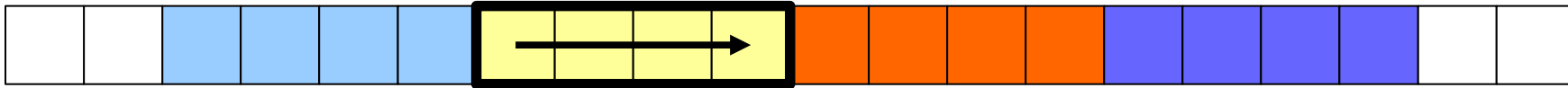
- Matrices in C are stored in *row-major order*
 - Elements of each row are contiguous in memory
 - Adjacent rows are contiguous in memory



Row-wise vs column-wise access



Row-wise access is OK: the data is contiguous in memory, so the cache helps (spatial locality)



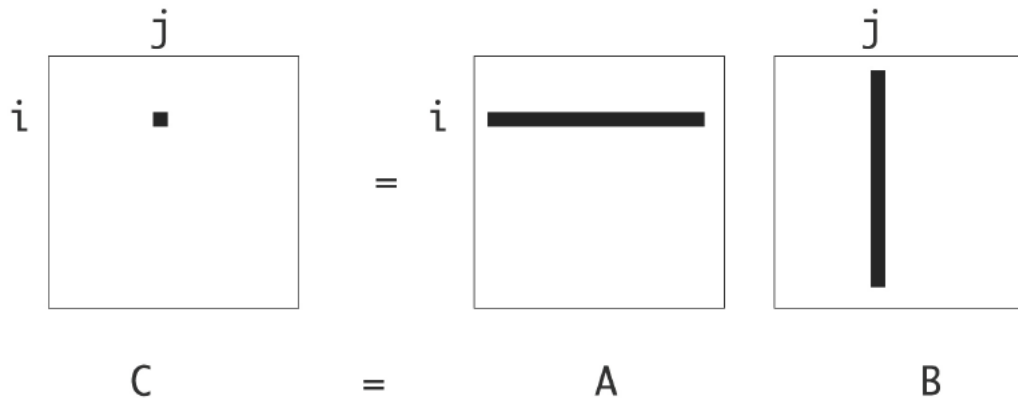
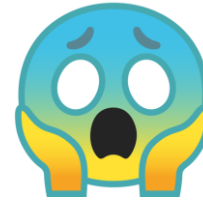
Column-wise access is **NOT OK**: the accessed elements are not contiguous in memory (*strided* access) so the cache does NOT help



Spatial Locality

Matrix-matrix product: $2n^3$ ops, $2n^2$ data (if we have $n \times n$ matrices)

- $c[i,j]$ is loaded and stored only once, thus a register
- $a[i,k]$ is accessed by row, so spatial locality (n accesses).
- $b[k,j]$, loaded every time, so $n^2 \times n$ times



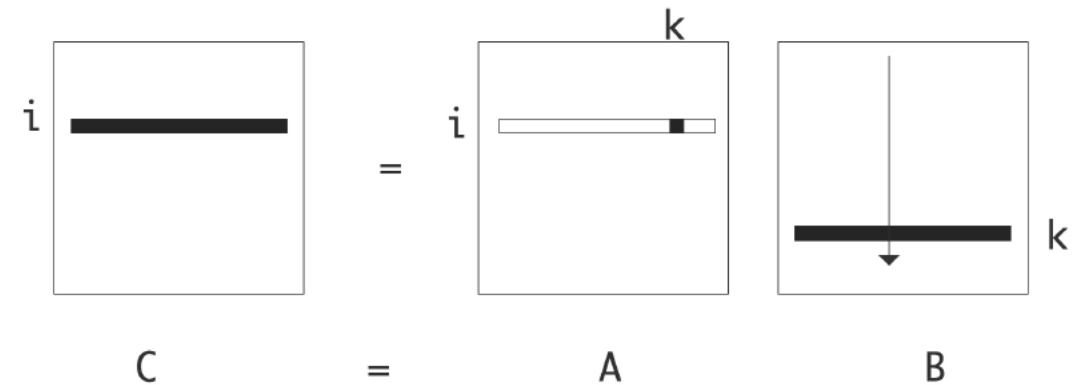
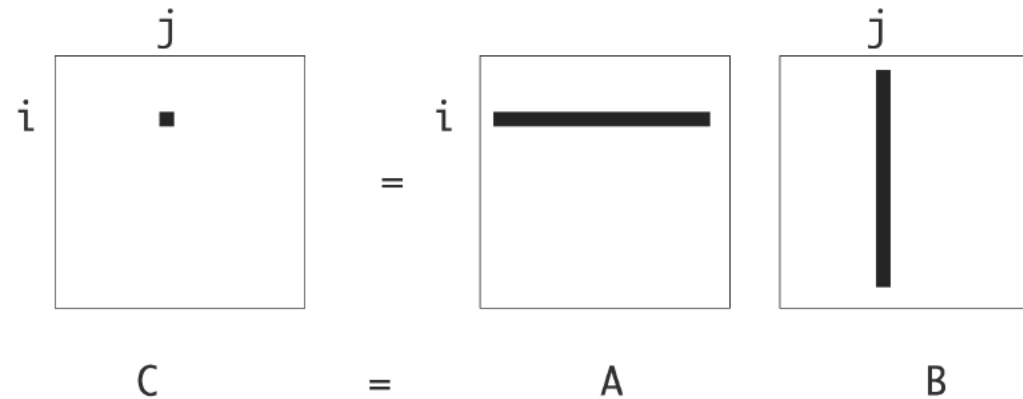
```
for i=1..n
  for j=1..n
    for k=1..n
       $c[i,j] += a[i,k] * b[k,j]$ 
```

Better algorithm



```
for i=1..n
  for j=1..n
    for k=1..n
      c[i,j] += a[i,k]*b[k,j]
```

```
for i=1..n
  for k=1..n
    for j=1..n
      c[i,j] += a[i,k]*b[k,j]
```



- $c[i,j]$ now requires n^3 **store** operation wrt n^2
- A and b exploits spatial and temporal locality

Loop tiling

FROM

```
for (i=0; i<n; i++)  
    ...
```

TO

```
bs = ... /* the blocksize */  
nblocks = n/bs /* assume that n is a multiple of bs */  
for (b=0; b<nblocks; b++)  
    for (i=b*bs, j=0; j<bs; i++, j++)  
        ...
```

WHY?

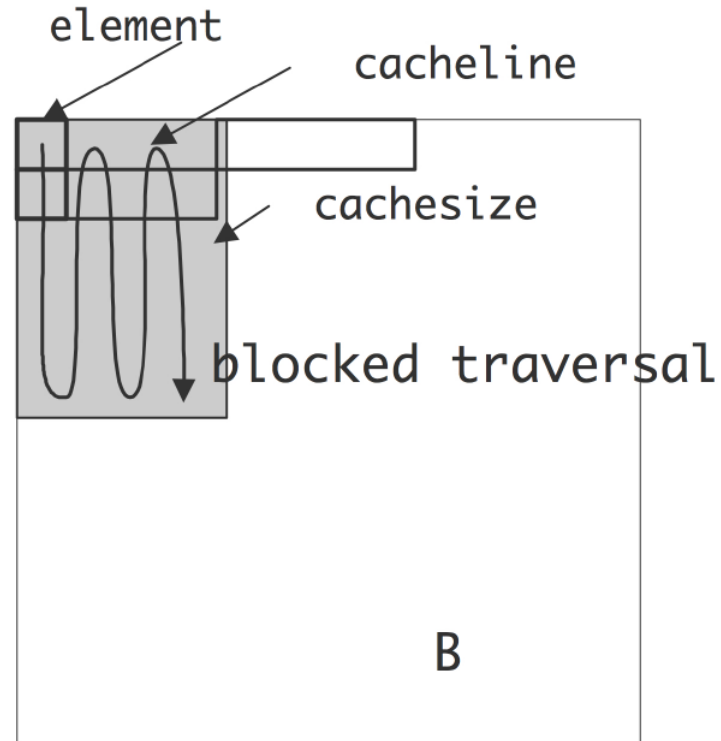
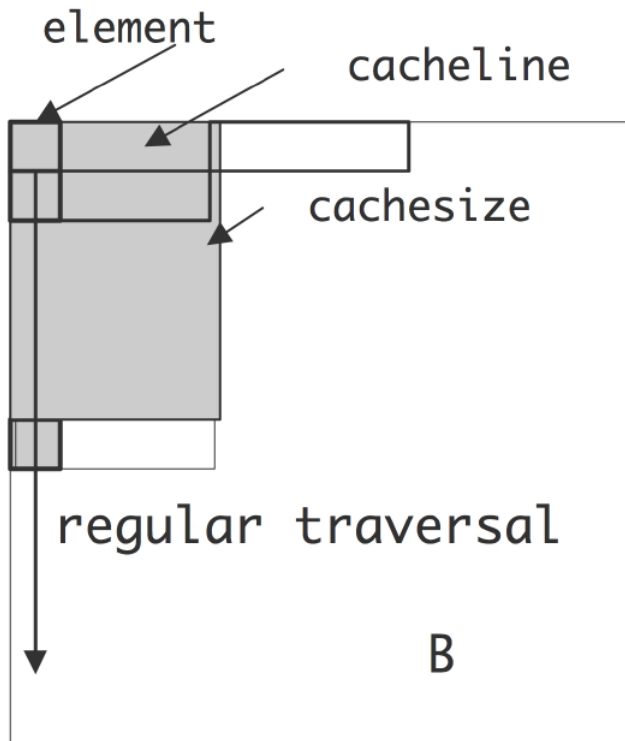
Sometimes performance can be increased by breaking up a loop into two nested loops, an outer one for the blocks in the iteration space, and an inner one that goes through the block. This is known as loop tiling: the (short) inner loop is a tile, many consecutive instances of which form the iteration space.

Because a data structure (i.e. array, matrix) is divided into blocks that fit in cache!

Example: matrix transposition

```
for (int i=0; i<N; i++)  
  for (int j=0; j<N; j++)  
    A[i][j] += B[j][i];
```

```
for (int ii=0; ii<N; ii+=blocksize)  
  for (int jj=0; jj<N; jj+=blocksize)  
    for (int i=ii*blocksize; i<MIN(N, (ii+1)*blocksize); i++)  
      for (int j=jj*blocksize; j<MIN(N, (jj+1)*blocksize); j++)  
        A[i][j] += B[j][i];
```



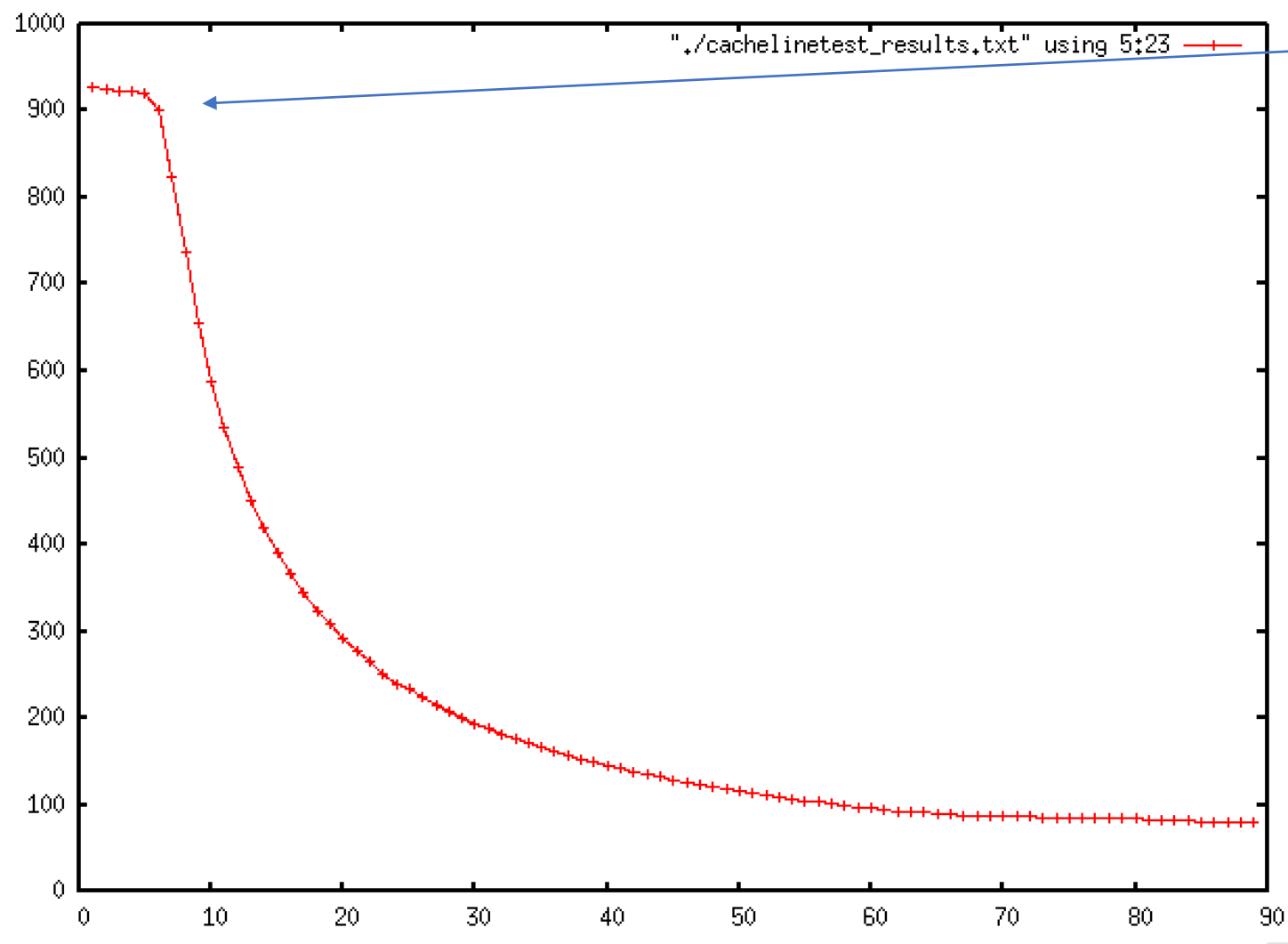
And the matrix-matrix multiplication?

Have a good read!

- <https://scialert.net/fulltextmobile/?doi=rjit.2011.61.67>
- <http://www.cs.utexas.edu/users/pingali/CS378/2008sp/papers/gotoPaper.pdf>

Let's try a test

- Compile and run the code in cacheline*.c
gcc -fopenmp -O2
- Plot a graph using gnuplot as follows:-
- Launch gnuplot which should return a prompt
- Issue the following line
plot "cachelinetest_results.txt" using 5:23 with linespoints
plot "./cachelinetest_double_results.txt" using 5:23 with linespoints
plot "./cachesizebwidth.txt" using 4:22 with linespoints
- where does the graph flatten out?



"./cachelinetest_results.txt" using 5:23

We use L1

Caches become useless

Conclusions

- A basic knowledge of the computers' architecture is fundamental for understanding the performance achievable by an application
- ILP is the first kind of parallelism we can exploit in sequential applications
- A proper cache exploitation is a fundamental factor for every application
- BUT the main solution for achieving high performance is represented by the following two kinds of parallelism, data parallelism and task parallelism.